

TEMA 5

JAVASCRIPT EN EL SERVIDOR: NODE.JS

APLICACIONES WEB - GIS - CURSO 2021/22



Esta obra está bajo una
Licencia CC BY-NC-SA 4.0 Internacional.

Jesús Cáceres Tello
Dpto de Sistemas Informáticos y Computación
Facultad de Informática
Universidad Complutense de Madrid

Este documento está basado en los materiales de **Manuel Montenegro** y **Marina de la Cruz**
bajo una licencia CC BY-NC-SA 4.0 Internacional

CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



INTRODUCCIÓN

FRONT-END



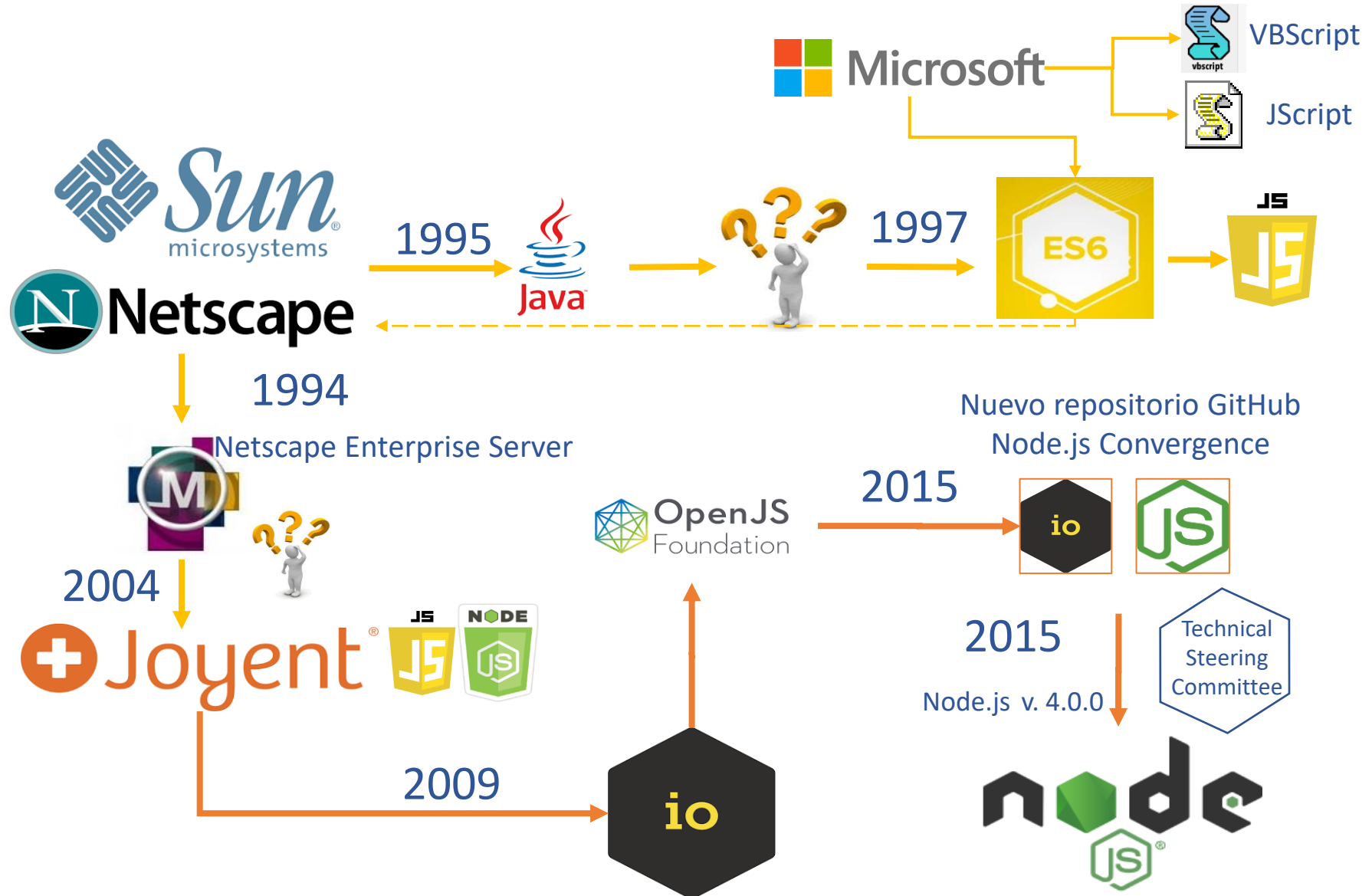
HTTP

HTTPS

BACK-END



INTRODUCCIÓN: Orígenes Node.js



INTRODUCCIÓN: node.js vs. io.js



- Interés claro a largo plazo (propiedad de Joyent)
- Soporte completo con foros de seguidores y gestores de paquetes
- Era una marca ampliamente conocida y reconocida.
- Versionado muy lento



- Soporte para las últimas versiones de V8 y ES6
- Desarrollo activo haciendo liberaciones semanales
- Integración continua y ejecución de pruebas al 100%
- Vinculación con la comunidad
- Gobernabilidad abierta por la comunidad y no por empresas
- Hoja de ruta predecible
- Versiones compatibles con SemVer
- Comienza a incrementar su actividad nuevamente con lanzamientos de versiones más rápidos utilizando, compatibles con SemVer, **versionado semántico**.



INTRODUCCIÓN: The OpenJS Foundation



- Su misión es la de fomentar el buen estado y crecimiento de JavaScript, Node.js y otros
- Certificaciones en Node.js
- Desarrollos colaborativos de JavaScript y otras tecnologías web
- [Appium](#), [Dojo](#), [Electron](#), [jQuery](#), [webpack](#)

INTRODUCCIÓN: V8 Engine

Node.js es una plataforma Javascript basada en el motor de Google V8 (C++)

Primera versión lanzada en 2008

Permite la ejecución de código en el lado del servidor.

Programáticamente es una caja negra.

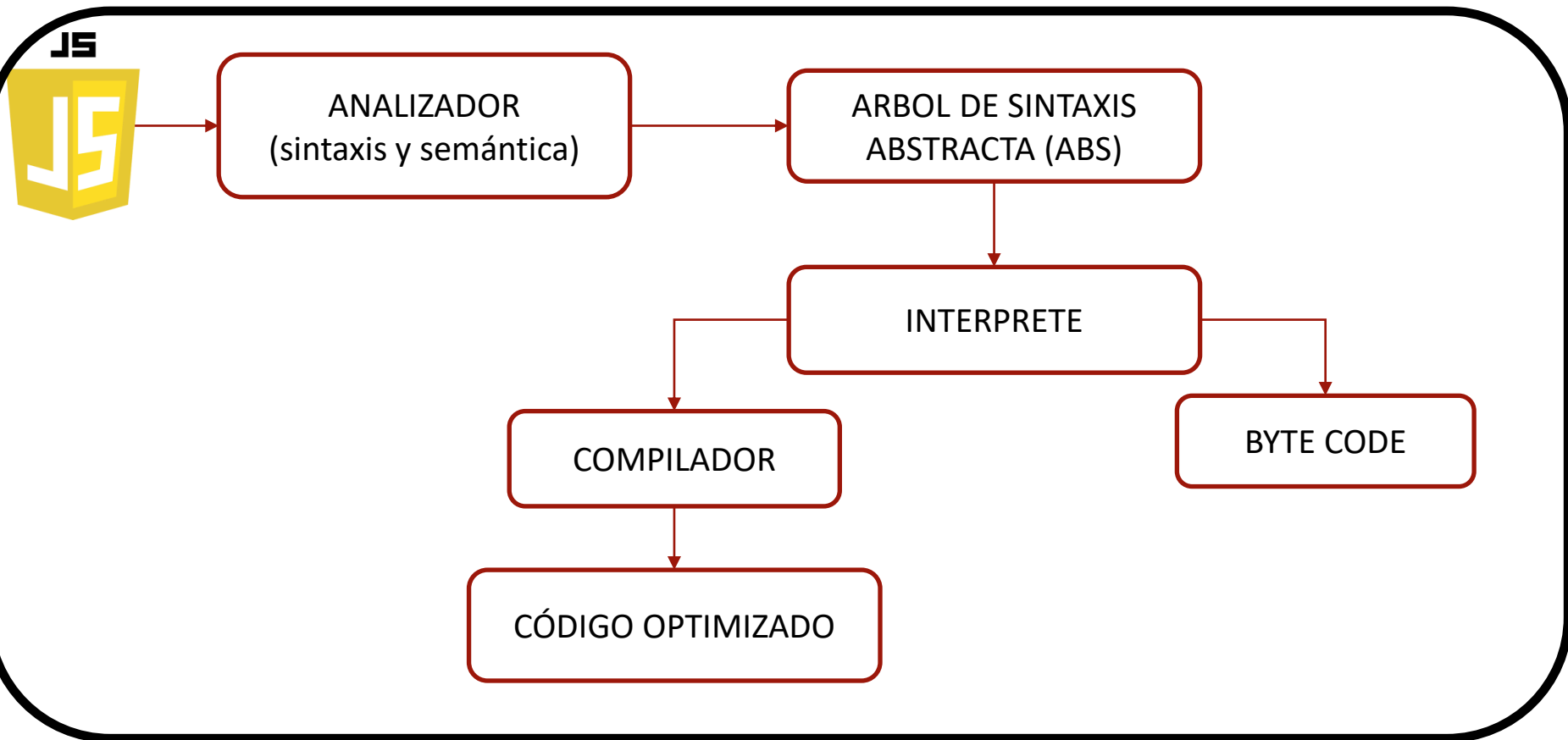
Características:

- Es Opensource
- Es independiente del navegador del cliente.
- Es manipulable, se puede descargar y modificar
- Está escrita en C++
- Es embebido, se puede utilizar en otros proyectos
- Es muy estable y se ha convertido en el motor de conversión más fiable (Edge, Chrome, Firefox, NodeJS)



INTRODUCCION: V8 Engine

- Compilación del código JavaScript en un hilo secundario lo que supone una reducción del tiempo de compilación entre un 5% y un 20%



INTRODUCCIÓN: V8 Engine

Node.js (o simplemente Node) es un entorno de ejecución que permite ejecutar **Javascript** fuera del navegador.

<https://nodejs.org/>

Se distribuye bajo una [licencia MIT](#).

Principalmente dirigido a servidores web, pero puede utilizarse para implementar cualquier tipo de aplicación.

En este curso utilizaremos Node para implementar la funcionalidad del **lado del servidor** en las aplicaciones web.

VERSIONES

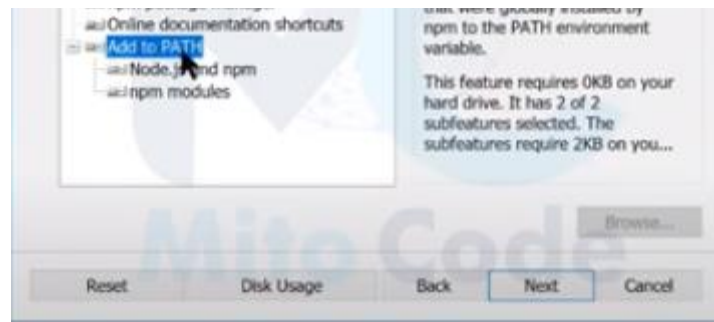
Versión	Nombre	Fecha de liberación	Estado LTS	Inicio LTS	Inicio de mantenimiento	Fin de mantenimiento
v0.10.x		2013-03-11	Sin soporte	-	2015-10-01	2016-10-31
v0.12.x		2015-02-06	Sin soporte	-	2016-04-01	2016-12-31
4.x	Argon ¹³	2015-09-08	Sin soporte	2015-10-01	2017-04-01	2018-04-30
5.x		2015-10-29	✗ No LTS	N/D		2016-06-30
6.x	Boron ¹³	2016-04-26	Sin soporte	2016-10-18	2018-04-30	Abril de 2019
7.x		2016-10-25	✗ No LTS	N/D		2017-06-30
8.x	Carbon ¹³	2017-05-30	Sin soporte	2017-10-31	2019-01-01 ¹⁴	Diciembre de 2019
9.x		2017-10-01	✗ No LTS	N/D		2018-06-30
10.x	Dubnium ¹³	2018-04-24	Sin soporte	30 de octubre de 2018	19 de mayo de 2019	30 de abril de 2021
11.x		2018-10-23	✗ No LTS	N/D		1 de junio de 2019
12.x	Erbium ¹³	2019-04-23	Sin soporte	21 de octubre de 2019	20 de octubre de 2020	30 de abril de 2022
13.x	-	22 de octubre de 2019	✗ No LTS	-	1 de abril de 2020	1 de junio de 2020
14.x	Fermium ¹³	21 de abril de 2020	Mantenimiento LTS	20 de octubre de 2020	19 de octubre de 2021	30 de abril de 2023
15.x	-	22 de octubre de 2020	Sin soporte	-	1 de abril de 2021	1 de junio de 2021
16.x	Gallium ¹³	21 de abril de 2021	Mantenimiento	26 de octubre de 2021	18 de octubre de 2022	30 de abril de 2024
17.x	-	19 de octubre de 2021	Sin soporte	N/D		1 de junio de 2022
18.x	-	19 de abril de 2022	Activo	25 de octubre de 2022	18 de octubre de 2023	30 de abril de 2025
19.x	-	18 de octubre de 2022	Prevista	-	1 de abril de 2023	1 de junio de 2023
20.x	-	18 de abril de 2023	Prevista	24 de octubre de 2023	22 de octubre de 2024	30 de abril de 2026

INSTALACIÓN

Puede descargarse desde:

<https://nodejs.org/es/download/>

- Las versiones para Windows y Mac incluyen un instalador.
- Desde GNU/Linux se puede:
 - Utilizar el gestor de paquetes de la distribución
 - Utilizar **nvm** (<http://nvm.sh>), que permite mantener varias versiones de Node en un mismo sistema
 - Descargar los binarios desde la URL anterior.



EJECUCIÓN DE UN SCRIPT

Basta con teclear:

```
# node nombre_script.js
```

Si se desea ejecutar el intérprete (REPL):

```
# node
```

REPL = *Read-Eval-Print Loop*

El intérprete permite evaluar las expresiones Node que se introduzcan por teclado.

EJEMPLO DE SESIÓN CON EL INTÉRPRETE

```
C:\>node -v  
v14.17.6
```

```
C:\>npm -v  
6.14.15
```

C:\Windows\System32\cmd.exe

```
C:\>node  
Welcome to Node.js v14.17.6.  
Type ".help" for more information.  
> 5 + 6  
11  
> let x= 4 + 5  
undefined  
> console.log("Hola gente")  
Hola gente  
undefined  
> x  
9  
> function suma(x, y) { return x + y}  
undefined  
> suma(4, 5)  
9  
> .exit  
  
C:\>
```

COMANDOS DEL INTÉRPRETE

- `.exit`
Sale del intérprete.
- `.load script.js`
Ejecuta las sentencias del fichero dado.
- `.save script.js`
Guarda todas las expresiones evaluadas en un fichero.
- `.break`
Cancelar introducción de expresión actual. Útil cuando se queda algún paréntesis abierto en la entrada.

PROYECTOS NODE

El uso del ejecutable `node` para proyectos grandes resulta algo tediosa, porque estos requieren, entre otras cosas:

- Gestión de dependencias (librerías externas).
- Arranque y parada del sistema.

HERRAMIENTAS PARA GESTIONAR PROYECTOS

- Desde la línea de comandos: `npm`.
- Desde un IDE: *Visual Studio Code*, *Netbeans*.

NPM (NODE PACKAGE MANAGER)

Es una herramienta que se incluye en con las nuevas distribuciones de Node JS. Permite:

- Crear proyectos vacíos
- Gestionar las librerías de las que hace uso un proyecto, consultando una base de datos pública que indica desde dónde descargarla
- Gestionar scripts: arranque del sistema, parada, ejecución de tests, etc.

Más información: <https://www.npmjs.com>

Comandos de NPM

- access
- adduser
- audit
- bin
- bugs
- c
- cache
- ci
- cit
- clean-install
- clean-install-test
- completion
- config
- create
- ddp
- dedupe
- deprecate
- dist-tag,
- docs
- doctor
- edit
- explore
- fund
- get
- help
- help-search
- hook,
- i
- **init**
- install
- install-ci-test
- install-test
- it
- link
- list
- ln
- login
- logout
- ls
- org
- outdated
- owner
- pack
- ping
- prefix
- profile
- prune
- publish
- rb
- rebuild
- repo
- restart
- root
- run
- run-script
- s
- se
- search
- set
- shrinkwrap
- star
- stars
- **start**
- stop
- t
- team
- **test**
- token
- tst
- un
- uninstall
- unpublish
- unstar
- up
- update
- v
- version
- view
- whoami

CREACIÓN DE UN PROYECTO CON npm (I)

Ejecutar desde la línea de comandos:

```
C:\>npm init
```

Se solicitará información sobre los datos del proyecto:

```
...  
package name: proyecto1  
version: (1.0.0)  
description: Esto es un proyecto de prueba  
entry point: (index.js) app.js  
test command:  
git repository:  
keywords: ucm fdi aw node proyecto prueba  
author: Jesús Cáceres  
license: (ISC) MIT  
...
```

CREACIÓN DE UN PROYECTO CON npm (II)

Se creará un fichero `package.json` con el siguiente contenido:

```
{  
  "name": "proyecto1",  
  "version": "1.0.0",  
  "description": "Esto es un proyecto de prueba",  
  "main": "app.js",  
  "scripts": {  
    "test": "echo \\\"Error: no test specified\\\" && exit 1"  
  },  
  "keywords": [  
    "ucm",  
    "fdi",  
    "aw",  
    "node",  
    "proyecto",  
    "prueba"  
  ],  
  "author": "Jesús Cáceres",  
  "license": "MIT"  
}
```

CREACIÓN DE UN PROYECTO CON npm (III)

```
package.json > ...  
1  {  
2    "name": "proyecto1",  
3    "version": "1.0.0",  
4    "description": "Esto es un proyecto de prueba",  
5    "main": "prueba1.js",  
6    "scripts": {  
7      "test": "echo \"Error: no test specified\" && exit 1"  
8    },  
9    "keywords": [  
10     "ucm",  
11     "fdi",  
12     "aw",  
13     "node",  
14     "proyecto",  
15     "prueba"  
16   ],  
17   "author": "Jesús Cáceres",  
18   "license": "MIT"  
19 }
```

CREACIÓN DE UN PROYECTO CON npm (III)

- Atributo `scripts`: permite añadir comandos para arrancar el programa, detenerlo, etc.

Por ejemplo, podemos añadir:

```
"scripts": {  
  "test": "mocha",  
  "start": "node prueba1.js"  
},
```



`npm install mocha --save`

para poder arrancar el sistema mediante

```
# npm start
```

o ejecutar los test mediante

```
# npm test
```

CREACIÓN DE UN PROYECTO CON npm (IV)

Si no se introduce un nombre para repositorio Git, es posible declarar el paquete como privado añadiendo:

```
....  
"author": "Jesús Cáceres",  
"license": "MIT",  
"private": true,  
....
```

De este modo evitaremos *warnings* sobre la ausencia de un repositorio.

CREACIÓN DE UN PROYECTO CON npm (V)

También se puede crear un repositorio a partir del proyecto git para la generación de repositorios.



git init

Initialized empty Git repository in C:/04_NODEJS/.git/

git status

On branch master

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)
nothing added to commit but untracked files present (use
"git add" to track)

```
Changes to be committed:
  (use "git rm --cached <file>..." to u
    new file:   node_modules/.bin/_
    new file:   node_modules/.bin/_
    new file:   node_modules/.bin/_
    new file:   node_modules/.bin/f
    new file:   node_modules/.bin/f
    new file:   node_modules/.bin/h
    new file:   node_modules/.bin/h
    new file:   node_modules/.bin/j
    new file:   node_modules/.bin/j
    new file:   node_modules/.bin/j
    new file:   node_modules/.bin/m
    new file:   node_modules/.bin/m
    new file:   node_modules/.bin/n
    new file:   node_modules/.bin/n
```

IDE: VISUAL STUDIO CODE

<https://code.visualstudio.com>

Es un IDE multiplataforma con licencia GPL.

Incorpora herramientas para la edición de código en Javascript y la gestión de proyectos en Node.



IDE: VISUAL STUDIO CODE

```
prueba1.js X
prueba1.js
1 console.log("Universidad Complutense");
```

PROBLEMS OUTPUT **TERMINAL** DEBUG CONSOLE

```
PS C:\Users\JCT014\Desktop\FORMACION\01_COMPLUTENSE\01 AW (2021-2022)\05_EJEMPLOS\04_NODEJS> node prueba1.js
Universidad Complutense
PS C:\Users\JCT014\Desktop\FORMACION\01_COMPLUTENSE\01 AW (2021-2022)\05_EJEMPLOS\04_NODEJS> npm help

Usage: npm <command>

where <command> is one of:
  access, adduser, audit, bin, bugs, c, cache, ci, cit,
  clean-install, clean-install-test, completion, config,
  create, ddp, dedupe, deprecate, dist-tag, docs, doctor,
  edit, explore, fund, get, help, help-search, hook, i, init,
  install, install-ci-test, install-test, it, link, list, ln,
  login, logout, ls, org, outdated, owner, pack, ping, prefix,
  profile, prune, publish, rb, rebuild, repo, restart, root,
  run, run-script, s, se, search, set, shrinkwrap, star,
  stars, start, stop, t, team, test, token, tst, un,
  uninstall, unpublish, unstar, up, update, v, version, view,
  whoami

npm <command> -h quick help on <command>
npm -l display full usage info
npm help <term> search for help on <term>
npm help npm involved overview

Specify configs in the ini-formatted file:
  C:\Users\JCT014\.npmrc
or on the command line via: npm <command> --key value
Config info can be viewed via: npm help config

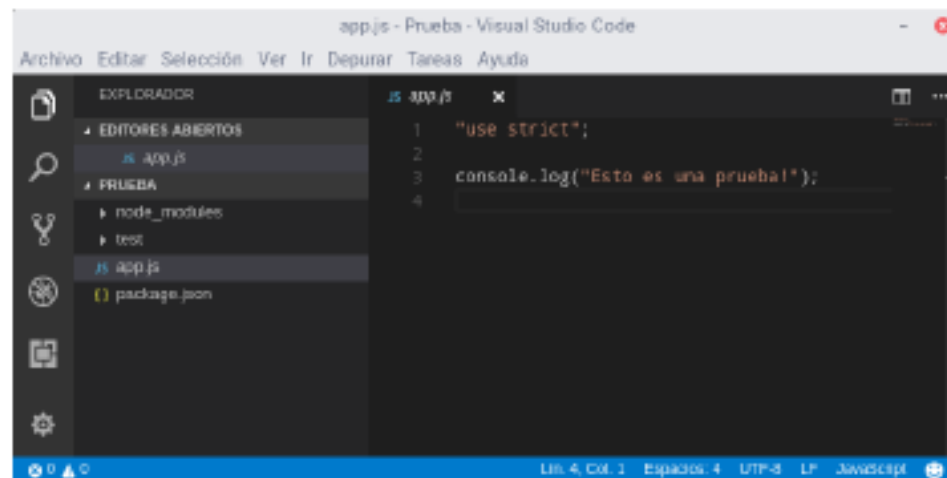
npm@6.14.15 C:\Program Files\nodejs\node_modules\npm
PS C:\Users\JCT014\Desktop\FORMACION\01_COMPLUTENSE\01 AW (2021-2022)\05_EJEMPLOS\04_NODEJS>
```

IDE: VISUAL STUDIO CODE

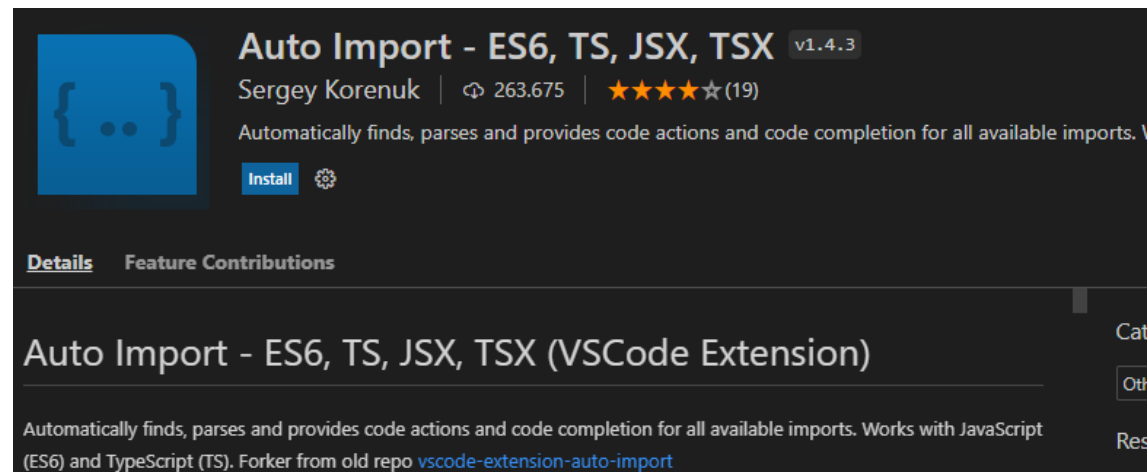
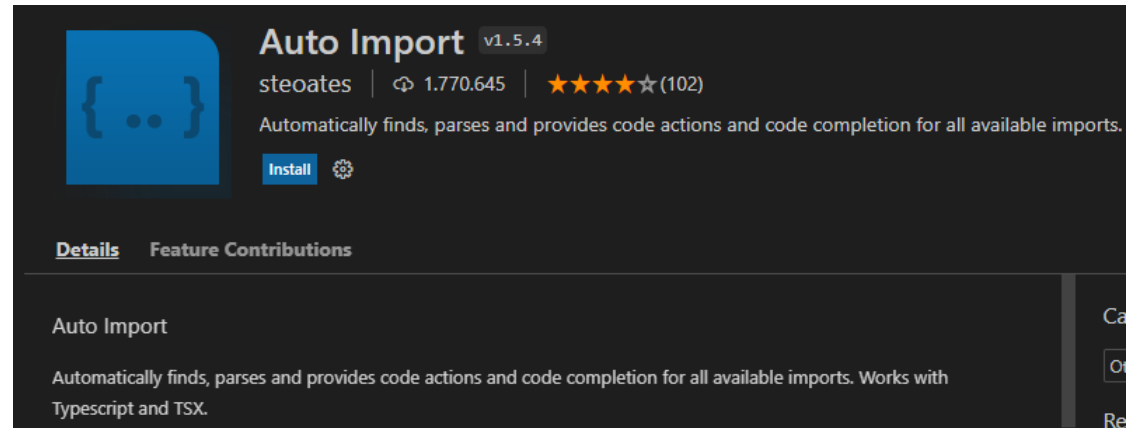
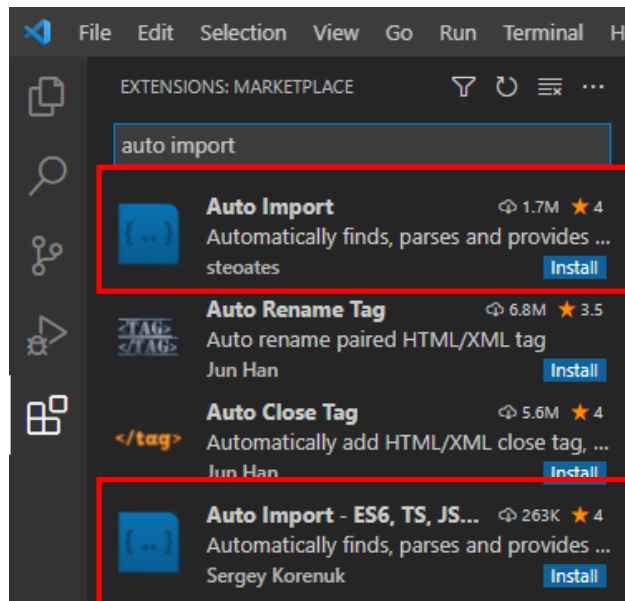
VS Code se apoya en **npm** para gestionar los proyectos Node.

Basta con abrir la carpeta donde se encuentre el fichero **package.json** de nuestro proyecto Node.

Archivo → Abrir carpeta... (Ctrl+K Ctrl+O)

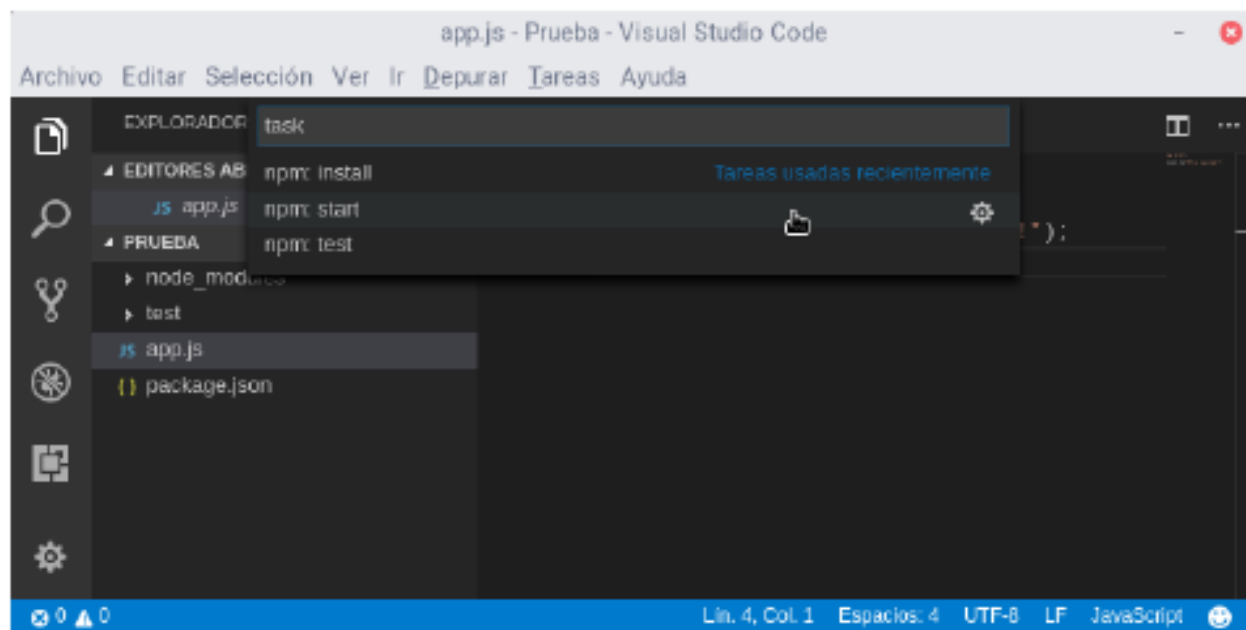


IDE: VISUAL STUDIO CODE



IDE: VISUAL STUDIO CODE

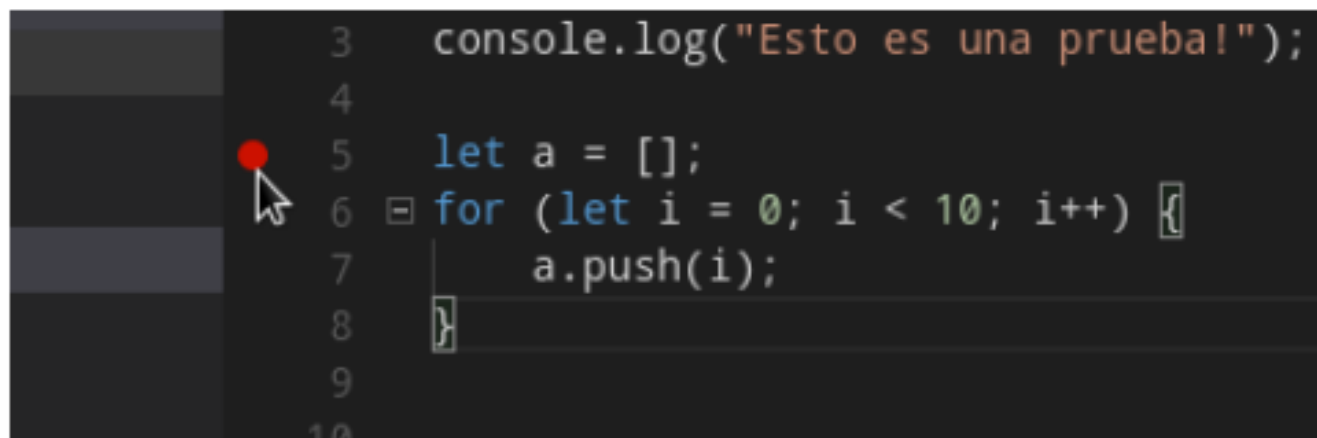
Podemos ejecutar el proyecto y los tests mediante la opción *Tareas* → *Ejecutar tarea*, o bien introduciendo en la paleta de comandos (*Ctrl+Shift+P*) la orden **task** seguida del nombre de la tarea a ejecutar:



IDE: VISUAL STUDIO CODE

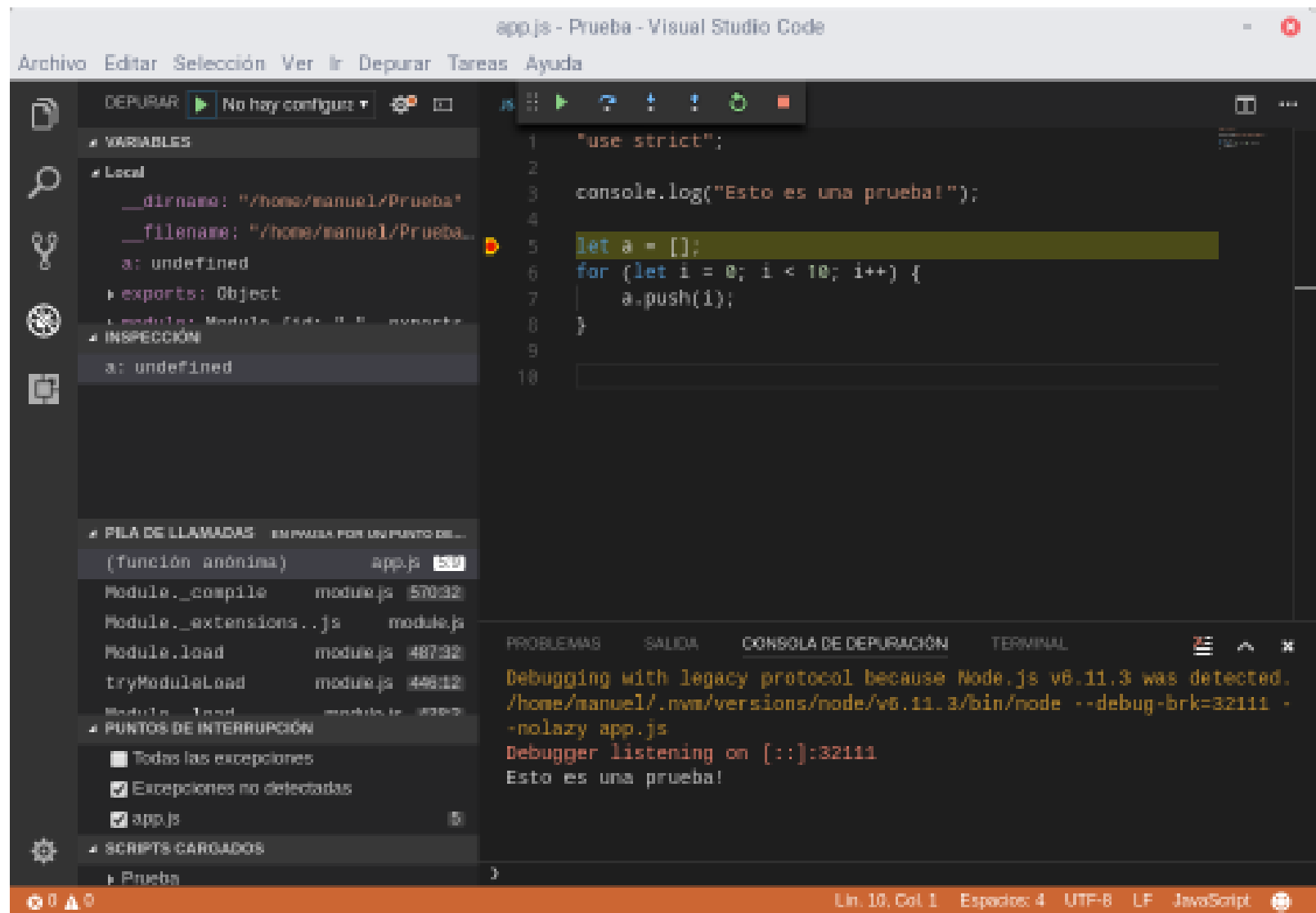
VS Code incluye un depurador de Javascript.

Introducimos puntos de ruptura utilizando el margen izquierdo del editor.



Después utilizamos la opción del menú *Depurar* → *Iniciar depuración* (F5)

IDE: VISUAL STUDIO CODE



IDE: APACHE NETBEANS

Netbeans dispone de un plugin para Node js.

Para trabajar con Node JS se requiere una versión igual o posterior a la 8.1.

Actualmente está disponible la versión 12



IDE: APACHE NETBEANS

Downloading Apache NetBeans 12.5

Apache NetBeans 12.5 was released September 13, 2021. See [Apache NetBeans 12.5 Features](#) for a full list of features.

Apache NetBeans 12.5 is available for download from your closest Apache mirror.

- Binaries: [netbeans-12.5-bin.zip](#) (SHA-512, PGP ASC)
- Installers:
 - [Apache-NetBeans-12.5-bin-windows-x64.exe](#) (SHA-512, PGP ASC)
 - [Apache-NetBeans-12.5-bin-linux-x64.sh](#) (SHA-512, PGP ASC)
 - [Apache-NetBeans-12.5-bin-macosx.dmg](#) (SHA-512, PGP ASC)



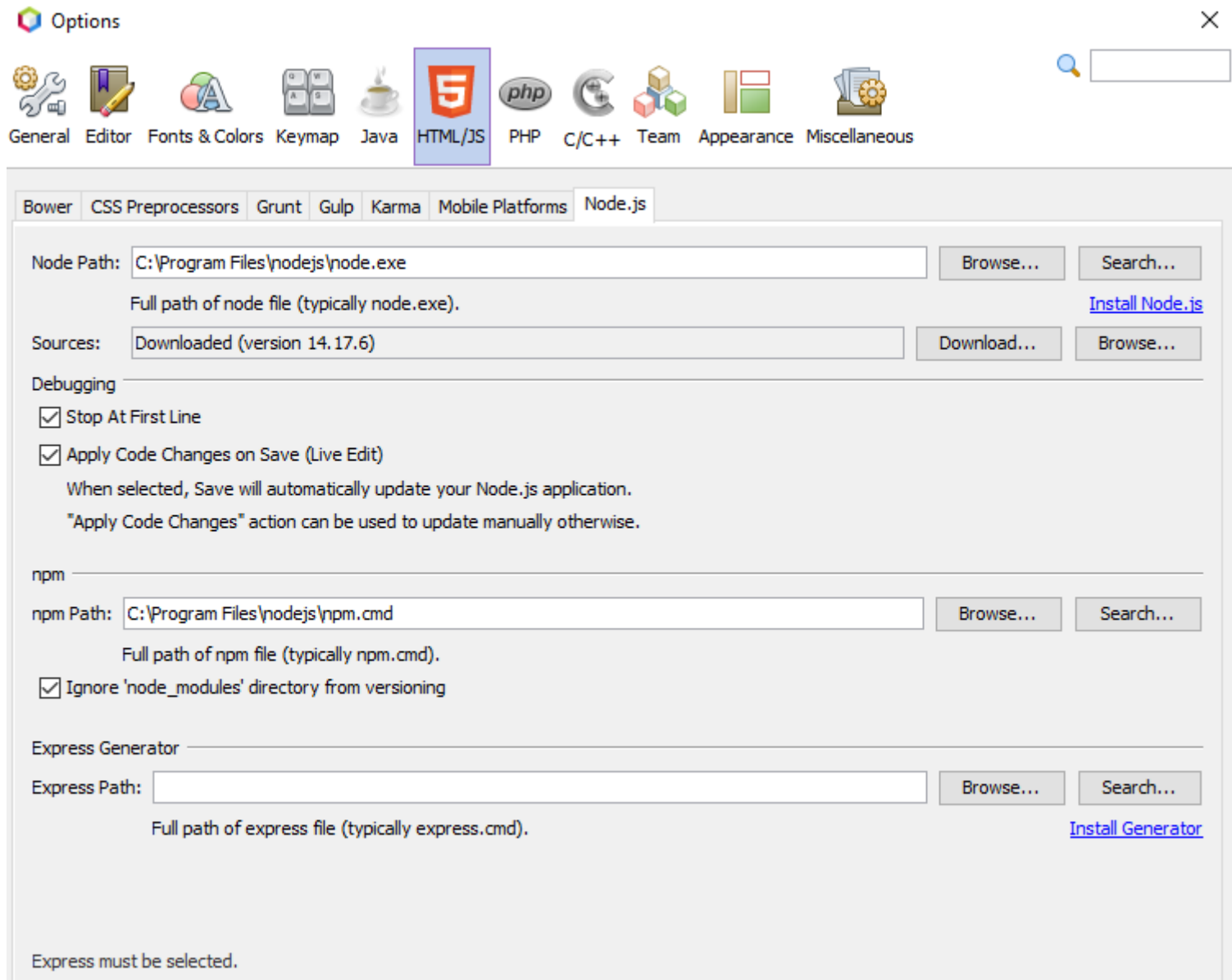
macOS versions prior to 10.14.4 require the [Swift 5 Runtime](#) to be installed to launch Apache NetBeans 12.5.

- Source: [netbeans-12.5-source.zip](#) (SHA-512, PGP ASC)
- Javadoc for this release is available at <https://bits.netbeans.org/12.5/javadoc>

Officially, it is important that you [verify the integrity](#) of the downloaded files using the PGP signatures (.asc files). The PGP keys used to sign this release are available [here](#).

Apache NetBeans can also be installed as a self-contained [snap package](#) on Linux.

IDE: APACHE NETBEANS



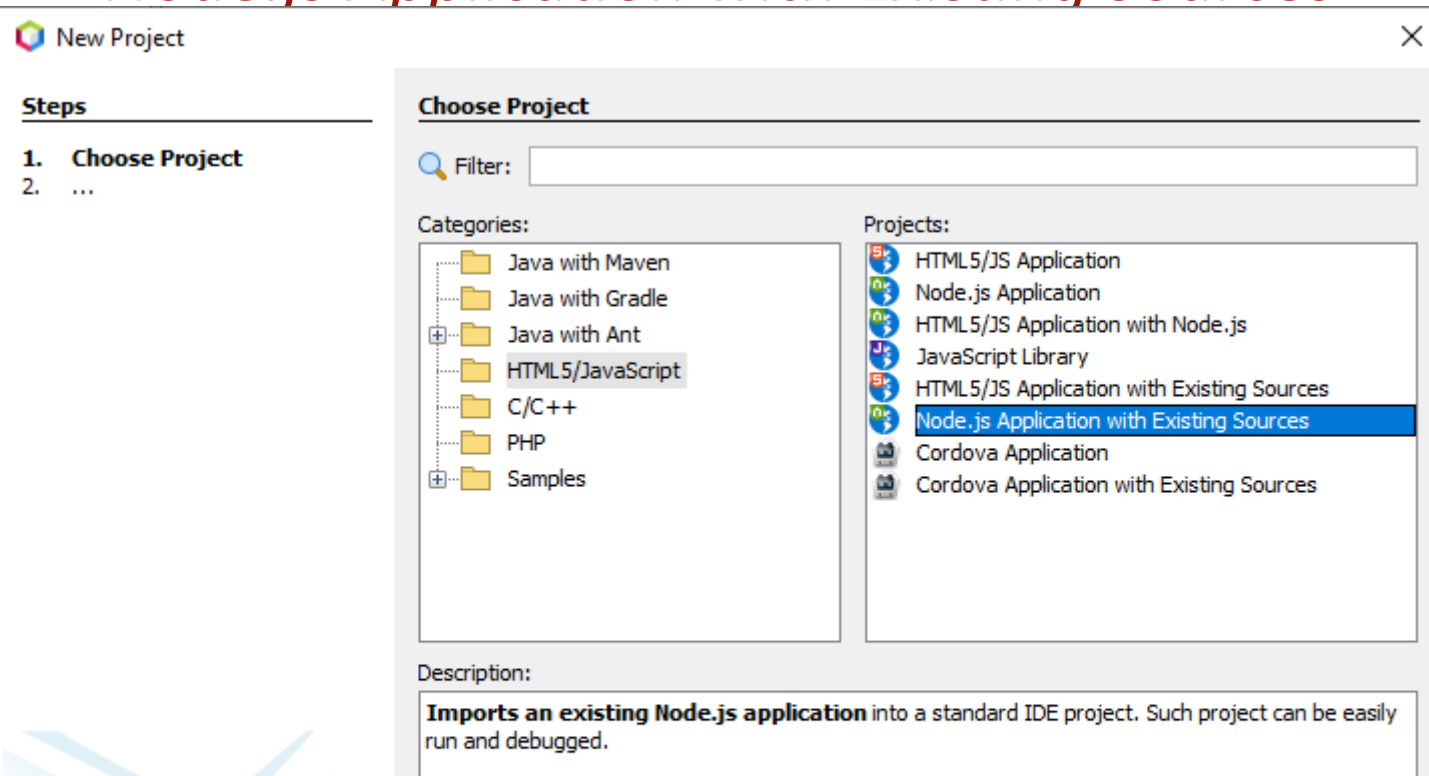
IDE: APACHE NETBEANS

File -> New Project ...

Seleccionar categoría *HTML5/JavaScript*

Y como tipo de aplicación

Node.js Application with Existing Sources



IDE: APACHE NETBEANS

 New Node.js Application with Existing Sources ✕

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Source Folder:

Site Root:

Project Name:

Project Directory:



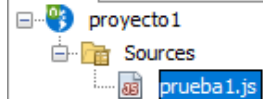
IDE: APACHE NETBEANS

proyecto1 - Apache NetBeans IDE 12.5

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help



Proj... X Files Services



prueba1.js X

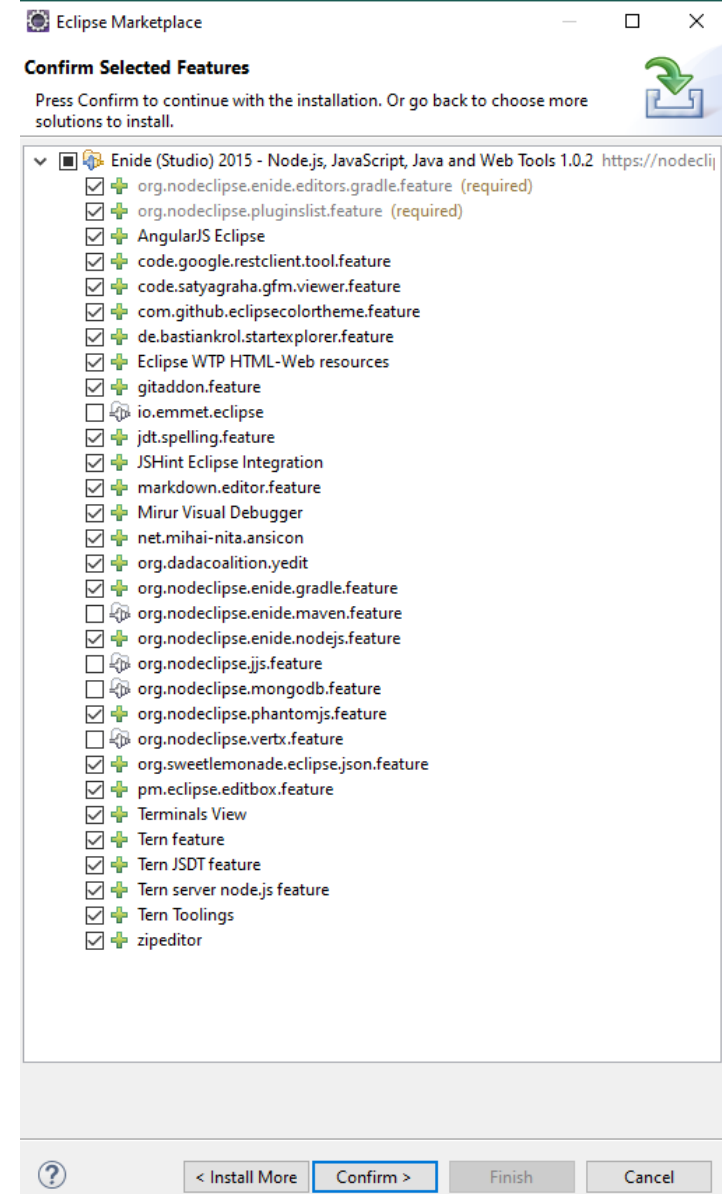
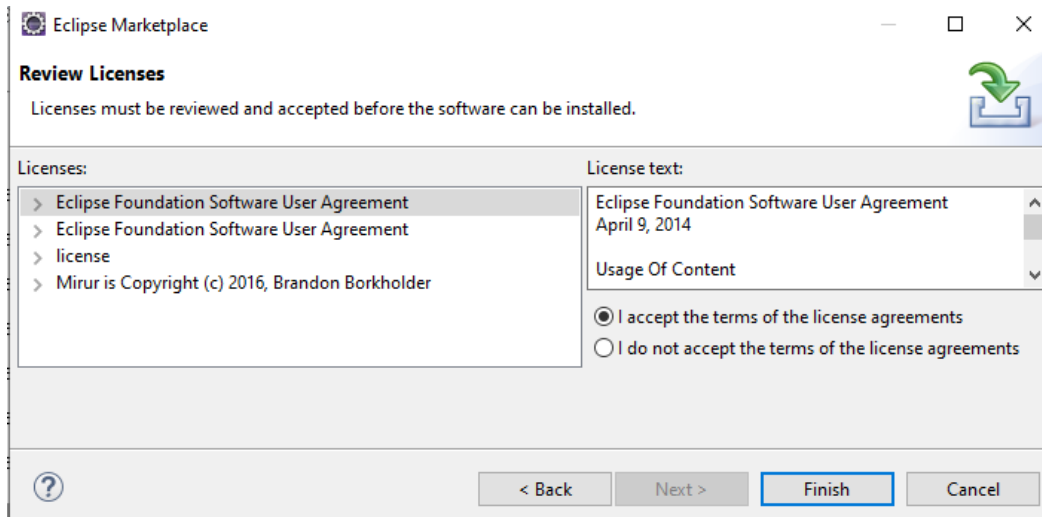
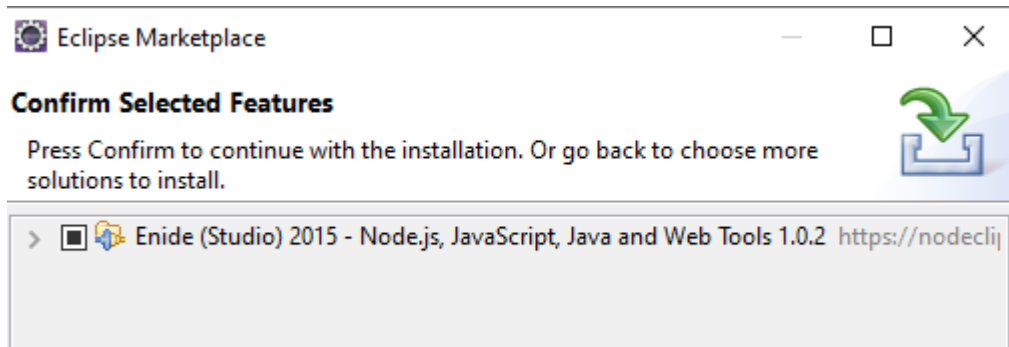
Source History

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this template
3   * Click nbfs://nbhost/SystemFileSystem/Templates/ClientSide/javascript.js to edit this template
4   */
5   console.log("Esto es una prueba");
6
7
```

Output - Node.js (proyecto1) X

```
"C:\Program Files\nodejs\node.exe" "C:\Users\JCT014\Documents\NetBeans_projects\MisProyectosNodeJS\prueba1.js"
Esto es una prueba
Done.
```

IDE: ECLIPSE



CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



MÓDULOS EN NODE JS

Los Módulos facilitan el desacoplamiento del código de un proyecto facilitando la productividad, el mantenimiento y la realización de test de código.

Existen 3 tipos de módulos:

- Módulos **fichero**:

Forman parte de un proyecto, pero no están pensados para ser reutilizados como librerías.

- Módulos **core**:

Vienen incluidos con Node. Proporcionan operaciones básicas (ficheros, servidores web, sistema operativo, etc.)

- Módulos **paquete**:

Son librerías externas, gestionadas por *npm*

MÓDULOS FICHERO CON UNA ÚNICA FUNCIÓN

Se crea un fichero JavaScript con la función a definir y ésta se asigna a la variable `module.exports`

```
1  "use strict"
2  function fibAux(n) {
3      if (n === 0) {
4          return 0;
5      } else if (n === 1) {
6          return 1;
7      } else {
8          return fibAux(n-1) + fibAux(n-2);
9      }
10 }
11
12 function fib(n) {
13     console.assert(typeof(n) === "number", `${n} is not a number`);
14     return fibAux(n);
15 }
16
17 module.exports = fib;
```


MÓDULOS FICHERO CON UNA ÚNICA FUNCIÓN

Suponemos que el código anterior está contenido en un fichero moduloFib.js

Ahora hacemos uso de la función fib desde otro fichero.

```
JS testModules.js > ...  
1  "use strict"  
2  const fib = require("../moduloFib.js");  
3  console.log(fib(10));
```

Si ejecutamos el testModules.js imprimirá:

```
55 05_EJEMPLOS\04_NODEJS> node testModules.js
```

MÓDULOS FICHERO CON UNA ÚNICA FUNCIÓN

La llamada a `require(modulo)`:

1. Ejecuta el fichero `modulo` pasado como parámetro.
2. Devuelve el objeto asignado a `module.exports` (en este caso, la función `fib`).

MÓDULOS FICHERO CON UNA ÚNICA FUNCIÓN

El objeto devuelto por **require** se guarda en una variable que no ha de coincidir necesariamente con el nombre de la función exportada.

```
const myFib = require("./moduloFib.js");  
  
console.log(myFib(10));
```

Esto es útil para prevenir conflictos de nombres entre los distintos módulos. Supongamos otro módulo **otroModuloFib.js** que también exporta una función **fib**.

```
const myFib = require("./moduloFib.js");  
const otroFib = require("./otroModuloFib.js");  
  
console.log(myFib(10));  
console.log(otroFib(10));
```

MÓDULOS FICHERO CON VARIAS FUNCIONES

EXPORTAR MÁS DE UNA FUNCIÓN

Hasta ahora hemos asignado una función a la variable `module.exports`.

Pero puede asignarse cualquier otro tipo de valor: enteros, arrays, objetos, etc.

Lo más frecuente es que un módulo quiera exportar varias funciones. Para ello se exporta un único **objeto** que contenga todas las funciones a exportar.

MÓDULOS FICHERO CON VARIAS FUNCIONES

```
function areaCuadrado(lado) {  
    return lado * lado;  
}  
function areaCirculo(radio) {  
    return Math.PI * radio * radio;  
}  
function perimetroCuadrado(lado) {  
    return 4 * lado;  
}  
function perimetroCirculo(radio) {  
    return 2 * Math.PI * radio;  
}  
  
module.exports = {  
    areaCuadrado: areaCuadrado,  
    areaCirculo: areaCirculo,  
    perimetroCuadrado: perimetroCuadrado,  
    perimetroCirculo: perimetroCirculo  
}
```

MÓDULOS FICHERO CON VARIAS FUNCIONES

```
module.exports = {  
  areaCuadrado: areaCuadrado,  
  areaCirculo: areaCirculo,  
  perimetroCuadrado: perimetroCuadrado,  
  perimetroCirculo: perimetroCirculo  
}
```

El objeto exportado contiene cuatro atributos, cada uno asociado a su función correspondiente.

Los nombres de estos atributos serán los que se utilicen cuando se haga uso de este módulo

MÓDULOS FICHERO CON VARIAS FUNCIONES

```
const geometria = require("./geometria.js");  
console.log(geometria.areaCuadrado(10));  
console.log(geometria.areaCirculo(10));
```

```
const g = require("./geometria.js");  
console.log(g.areaCuadrado(10));  
console.log(g.areaCirculo(10));
```

```
100  
314.1592653589793  
100  
314.1592653589793
```

MÓDULOS FICHERO CON VARIAS FUNCIONES

Otra posible forma de definir un módulo:
Definir directamente las funciones en el objeto
asignado a `module.exports`

```
module.exports = {  
  areaCuadrado: function(lado) {  
    return lado * lado;  
  },  
  
  areaCirculo: function(radio) {  
    return Math.PI * radio * radio;  
  },  
  
  perimetroCuadrado: function(lado) {  
    return 4 * lado;  
  },  
  
  perimetroCirculo: function(radio) {  
    return 2 * Math.PI * radio;  
  }  
}
```


MÓDULOS FICHERO CON VARIAS FUNCIONES

Lo que no se exporte dentro de un módulo se considera privado al mismo. Esto nos permite modificar la implementación sin alterar la interfaz.

```
// moduloFib.js
// -----

const PHI = (1 + Math.sqrt(5)) / 2;

// Cambio la función fibAux por esta:
function fibMasEficiente(n) {
  const p1 = Math.pow(PHI, n);
  const p2 = Math.pow(1 - PHI, n);
  return Math.round((p1 - p2) / Math.sqrt(5));
}

function fib(n) {
  console.assert(typeof(n) === "number",
    `fib: ${n} is not a number`);
  return fibMasEficiente(n);
}

module.exports = fib;
```

MÓDULOS FICHERO CON VARIAS FUNCIONES

IMPORTANTE

A la hora de importar un módulo de tipo fichero mediante **require**, es necesario indicar el **path** del fichero que se importa, **aunque se encuentre en el mismo directorio**.

```
// Correcto:  
const geometria = require("./geometria.js");  
  
// Correcto:  
const geometria = require("../aw/librerias/geometria.js");  
  
// Correcto:  
const geometria = require("../geometria.js");  
  
// Incorrecto:  
const geometria = require("geometria.js");
```

No obstante, puede omitirse la extensión **.js**

```
// Correcto:  
const geometria = require("./geometria");
```

MÓDULOS FICHERO CON VARIAS FUNCIONES

```
// Incorrecto  
const geometria = require("geometria.js");
```

Si no se especifica un path, Node entiende que se quiere importar un módulo otro tipo:

- Módulo core
- Módulo de tipo paquete

MÓDULOS CORE

Forman parte de la distribución de Node

No necesitan ser instalados

También se importan mediante **require**, pero **sin**
indicar un path relativo

```
const util = require("util");  
const mensaje = util.format("Estamos en el año %d", 2023);  
console.log(mensaje);  
  
const os = require("os");  
console.log('Nombre del host: ' + os.hostname());  
console.log('Directorio personal: ' + os.homedir());
```

Estamos en el año 2023

Nombre del host: 5CG0439V NK

Directorio personal: C:\Users\JCT014

MÓDULOS CORE: LISTA DE MÓDULOS

- **fs** : Permite trabajar con el sistema de archivos y directorios.
- **path**: Permite analizar rutas de archivos del sistema de una forma consistente en diferentes plataformas.
- **querystring**: Analiza los datos del query string.
- **net**: Permite trabajar con la red utilizando diferentes protocolos.
- **stream**: Módulo para hacer uso de data-streams (streams de datos).
- **events**: Módulo para implementar emisores de eventos (el patrón observador de node).
- **child_process**: Permite desahogar procesos externos.
- **os**: Permite acceder información al nivel del sistema operativo.
- **url**: Permite analizar direcciones URL.
- **http** y **https**: Permite realizar (cliente) peticiones y aceptar (servidor) consultas.
- **util**: Proporciona funciones para formatear cadenas, convertir objetos en cadenas, verificar el tipo de objetos y realizar escritura síncrona en secuencias de salida
- **assert**: Permite realizar tests basados en asserts.
- **crypto**: Módulo para encrypting y hashing.

MÓDULOS DE TIPO PAQUETE

Son módulos pensados para ser reutilizados e incorporados a otros proyectos.

Se instalan dentro la propia carpeta del proyecto, en una subcarpeta llamada `node_modules`.

La información de la mayoría de ellos se encuentra en una base de datos pública: *NPM Registry*.

Se importan con `require` y, al igual que los módulos core, no se indica el path relativo.

MÓDULOS DE TIPO PAQUETE

También se puede importar mediante **require**, pero **sin** indicar un path relativo

```
const osHomedir = require ("os-homedir");
console.log(osHomedir());

var table = require('text-table');
var t = table([
  [ 'master', '0123456789', 'abcdef' ],
  [ 'staging', 'fedcba', '9876543210' ]
]);
console.log(t);
```

Nombres de módulos core: **uri-js**, **text-table**, **slide**, **path-exist**, etc...

```
C:\Users\JCT014
master    0123456789  abcdef
staging   fedcba       9876543210
```

MÓDULOS DE TIPO PAQUETE

BÚSQUEDA DE UN MÓDULO DE TIPO PAQUETE

```
const foo = require("foo");
```

Se busca `foo` en los siguientes directorios:

- `node_modules/foo.js`
- `../node_modules/foo.js`
- `../../node_modules/foo.js`
- `../../../node_modules/foo.js`
- y así sucesivamente, hasta llegar al directorio raíz

MÓDULOS DE TIPO PAQUETE

PAQUETES CON VARIOS MÓDULOS

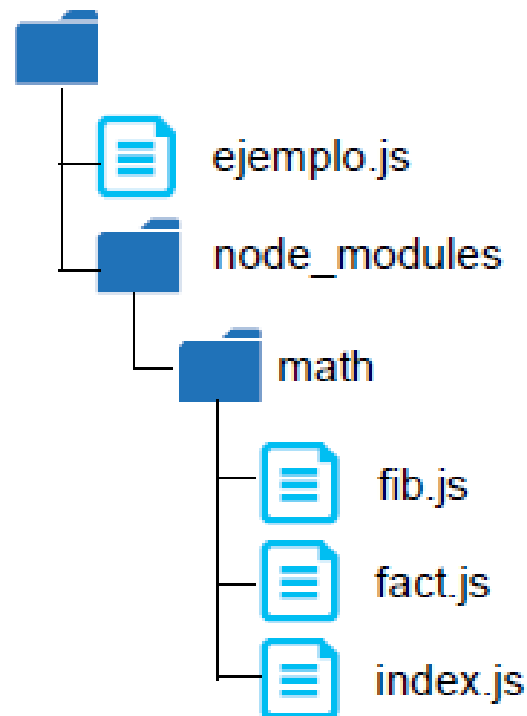
Un paquete puede contener más de un fichero `.js`

En este caso han de agruparse todos los `.js` en la misma carpeta, y guardar en esta carpeta un módulo `index.js` que exporte las funciones necesarias de cada uno de los módulos del paquete.

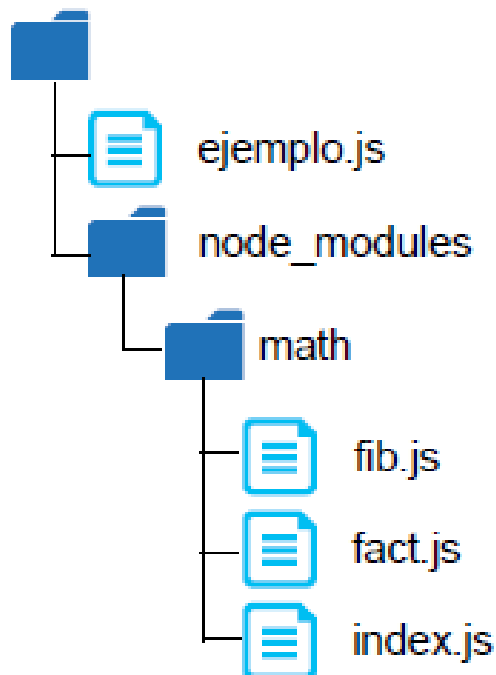
Para importar este paquete mediante `require` se debe indicar el nombre de la carpeta.

MÓDULOS DE TIPO PAQUETE

Partimos de esta estructura de directorios:



MÓDULOS DE TIPO PAQUETE



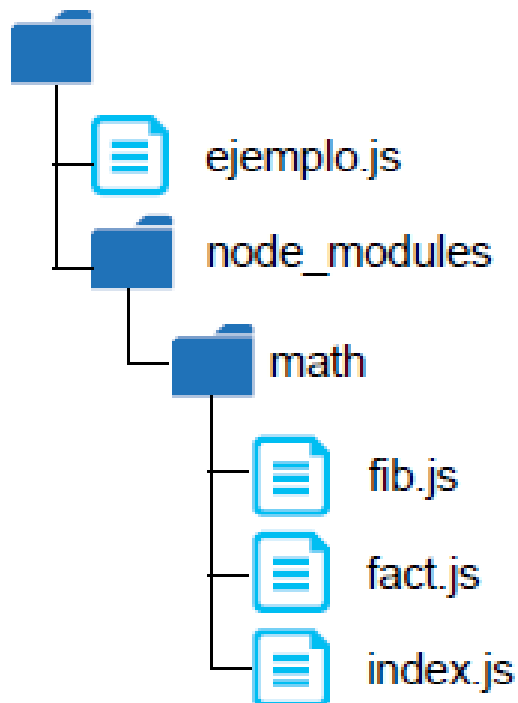
fib.js

```
"use strict";  
  
function fib(n) { ... }  
  
module.exports = fib;
```

fact.js

```
"use strict";  
  
function fact(n) { ... }  
  
module.exports = fact;
```

MÓDULOS DE TIPO PAQUETE

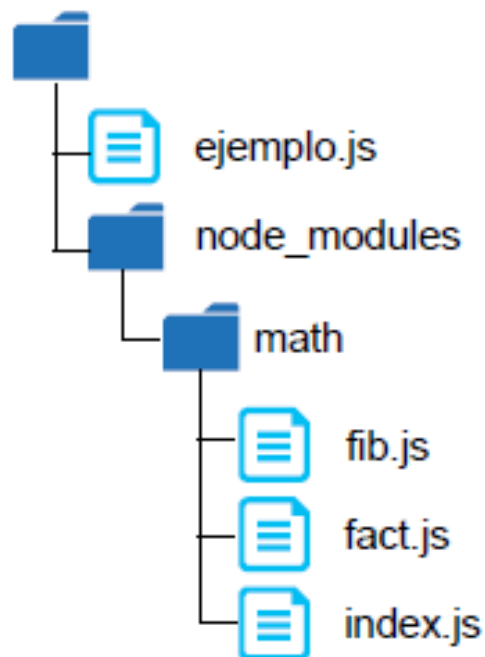


`index.js`

```
"use strict";

module.exports = {
  fib: require("../fib"),
  fact: require("../fact")
}
```

MÓDULOS DE TIPO PAQUETE



`ejemplo.js`

```
"use strict";  
  
const math = require("math");  
  
console.log(`fib(6) = ${math.fib(6)}`);  
console.log(`fact(6) = ${math.fact(6)}`);
```

Resultado:

```
$ node ejemplo.js  
fib(6) = 8  
fact(6) = 720
```

ÁRBOL DE DEPENDENCIAS

A menudo, la implementación de un proyecto Node requiere la instalación de unos determinados paquetes.

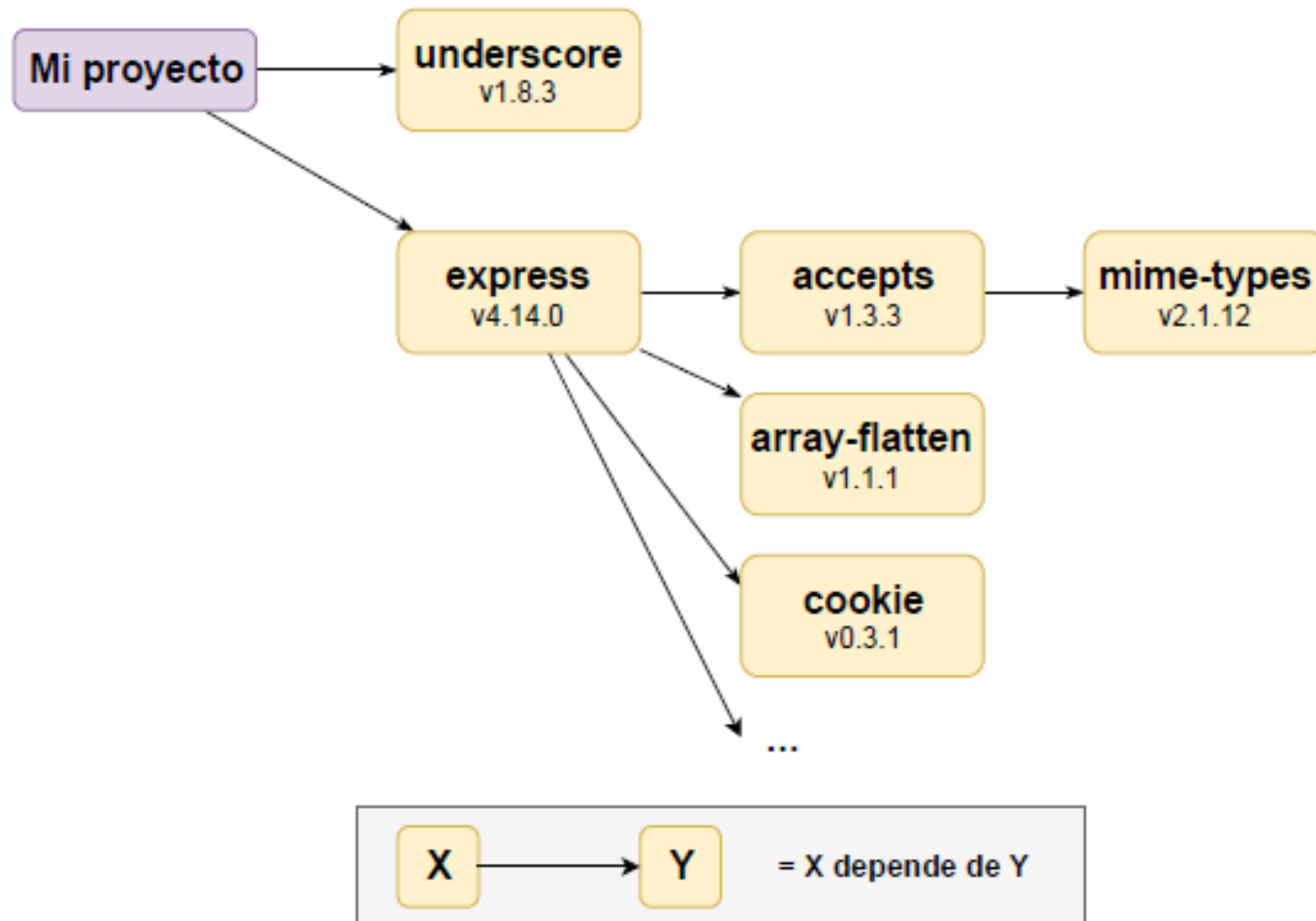
Cada uno de estos últimos pueden requerir la instalación de otros paquetes que, a su vez, requieren de otros paquetes.

Cuando la presencia del paquete X es un requisito para el funcionamiento del paquete Y, decimos que **Y depende de X**.

La relación de dependencia entre paquetes es transitiva, de modo que se tiene un **árbol de dependencias**.

ÁRBOL DE DEPENDENCIAS

EJEMPLO



INSTALACIÓN DE DEPENDENCIAS CON npm

La herramienta `npm` permite gestionar las dependencias de un proyecto. De hecho, `npm` fue inicialmente concebido como un instalador de paquetes.

Esta herramienta puede descargar paquetes desde un repositorio, e incorporarlos en nuestro proyecto.

<https://www.npmjs.com/>

INSTALACIÓN DE DEPENDENCIAS CON npm

Para descargar e instalar un paquete en un proyecto, ejecutar lo siguiente desde el directorio raíz del proyecto:

```
# npm install nombre_paquete
```

Por ejemplo, para incorporar el paquete **underscore** en nuestro proyecto:

```
# npm install underscore
```

Descargará la versión más reciente del paquete y la instalará bajo la carpeta **node_modules** del proyecto (creándola si es necesario).

INSTALACIÓN DE DEPENDENCIAS CON npm

El comando `npm install` no sólo instala el paquete especificado, sino también sus dependencias.

Por ejemplo al instalar `express`:

```
# npm install express
```

Se instala el paquete `express` y todo su árbol de dependencias bajo la carpeta `node_modules`:

```
node_modules
├── express
├── accepts
├── array-flatten
├── content-disposition
├── content-type
├── cookie
└── ...
```

INCLUIR DEPENDENCIAS EN package.json

Podemos (y es recomendable) especificar las dependencias de un proyecto en su fichero `package.json` mediante la propiedad `dependencies`:

```
"license": "MIT",  
"private": true,  
"dependencies": {  
  "mocha": "^9.1.3",  
  "os-homedir": "^2.0.0",  
  "text-table": "^0.2.0"  
},
```

INCLUIR DEPENDENCIAS EN `package.json`

Por defecto, `npm` no modifica el fichero `package.json` tras añadir un paquete a nuestro proyecto.

No obstante, al instalar un paquete podemos indicar a `npm` que añada la dependencia correspondiente a `package.json`. Esto se hace mediante la opción `--save`.

```
# npm install underscore --save  
# npm install express --save
```

INCLUIR DEPENDENCIAS EN `package.json`

INSTALACIÓN GLOBAL DE PAQUETES

A veces un paquete incorpora ficheros ejecutables diseñados para su uso en cualquier proyecto del sistema (`mocha`, `nodemon`, `grunt`, `jshint`, etc.)

Para poder ser invocados desde cualquier proyecto, estos ejecutables han de instalarse de manera global.

La instalación de un paquete de forma global puede requerir **permisos de administrador** en el sistema.

INCLUIR DEPENDENCIAS EN `package.json`

Para instalar un paquete de manera global se ha de especificar la opción `-g` a `npm`.

Por ejemplo:

```
# npm install -g nodemon
```

DISTRIBUCIÓN DE UN PROYECTO

Cuando se quiere distribuir un proyecto o publicar en un repositorio (p.ej. *GitHub*), **no es necesario** incluir el directorio **node_modules** con las dependencias.

Basta con indicar las dependencias en el **package.json**, y distribuir este último junto con el proyecto.

Mediante el comando:

```
# npm install
```

se reconstruirá la carpeta **node_modules** a partir de la información contenida en **package.json**, descargando los paquetes que sean necesarios.

OTRAS OPCIONES DE npm

- `npm ls`
Muestra el árbol de dependencias del proyecto actual.
- `npm search term`
Busca los paquetes del repositorio *node* cuyo nombre contenga `term`.
- `npm update nombre_paquete`
Actualiza un paquete a la última versión disponible.
- `npm dedupe`
Reorganiza la carpeta `node_modules` para evitar duplicidades.
- `npm publish`
Sube los datos del proyecto actual en el *NPM Registry*.

LISTAR ÁRBOL DE DEPENDENCIAS

Listar los módulos instalados en la carpeta `node_modules`

`npm list`

`npm list --depth {0..n}`

```
+-- @ungap/promise-all-settled@1.1.2
+-- ansi-colors@4.1.1
+-- browser-stdout@1.3.1
+-- chokidar@3.5.2
| +-- anymatch@3.1.2
| | +-- normalize-path@3.0.0 deduped
| | `-- picomatch@2.3.0
+-- braces@3.0.2
| `-- fill-range@7.0.1
|   `-- to-regex-range@5.0.1
|     `-- is-number@7.0.0
+-- UNMET OPTIONAL DEPENDENCY fsevents@2.3.2
+-- glob-parent@5.1.2
| `-- is-glob@4.0.3 deduped
+-- is-binary-path@2.1.0
| `-- binary-extensions@2.2.0
+-- is-glob@4.0.3
| `-- is-extglob@2.1.1
+-- normalize-path@3.0.0
  `-- readdirp@3.6.0
    `-- picomatch@2.3.0 deduped
+-- debug@4.3.2
| `-- ms@2.1.2
|   `-- diff@5.0.0
```

```
-- mocha@9.1.3
+-- @ungap/promise-all-settled@1.1.2
+-- ansi-colors@4.1.1
+-- browser-stdout@1.3.1
+-- chokidar@3.5.2
+-- debug@4.3.2
+-- diff@5.0.0
+-- escape-string-regexp@4.0.0
+-- find-up@5.0.0
+-- glob@7.1.7
+-- growl@1.10.5
+-- he@1.2.0
+-- js-yaml@4.1.0
+-- log-symbols@4.1.0
+-- minimatch@3.0.4
+-- ms@2.1.3
+-- nanoid@3.1.25
+-- serialize-javascript@6.0.0
+-- strip-json-comments@3.1.1
+-- supports-color@8.1.1
+-- which@2.0.2
+-- workerpool@6.1.5
+-- yargs@16.2.0
+-- yargs-parser@20.2.4
  `-- yargs-unparser@2.0.0
```

CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



MODELO ASÍNCRONO

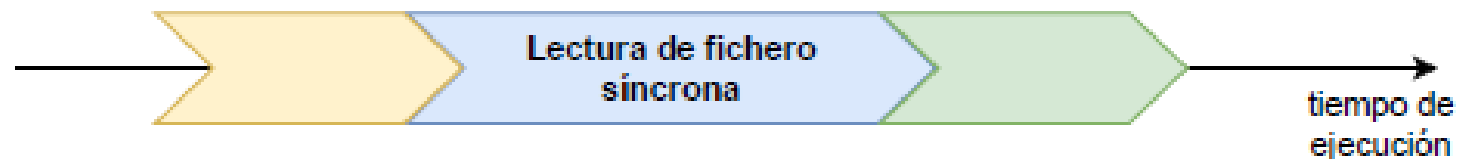
Node utiliza un modelo de entradas y salidas **asíncronas** con el fin de permitir el desarrollo de aplicaciones altamente concurrentes.

Este modelo resulta muy útil en un contexto de aplicaciones web, donde un servidor web ha de procesar miles de peticiones por segundo.

MODELO SÍNCRONO vs ASÍNCRONO

Cuando ejecutamos una operación **síncrona**, la ejecución del programa **se detiene** hasta que dicha operación haya terminado.

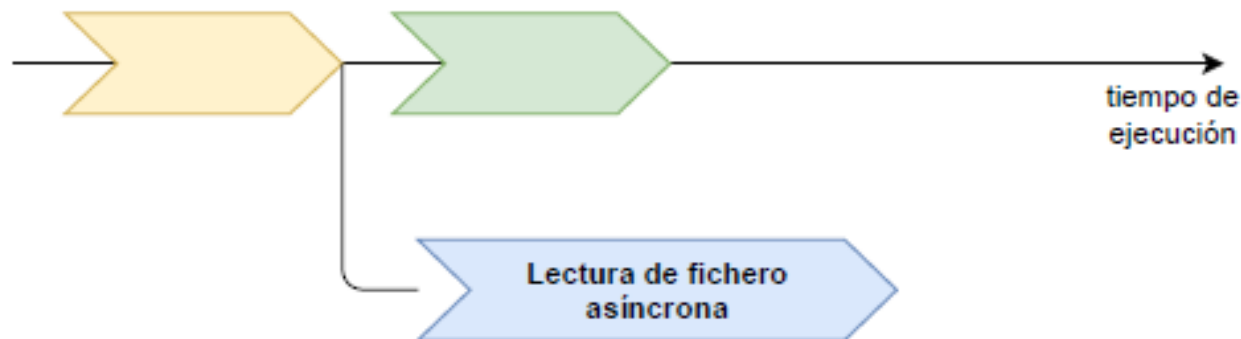
Ej: Lectura de fichero síncrona



MODELO SÍNCRONO vs ASÍNCRONO

Cuando ejecutamos una operación **asíncrona**, la ejecución del programa **continúa**, mientras la operación se realiza de manera «**concurrente**».

Ej: Lectura de fichero asíncrona



EJEMPLO: LECTURA DE FICHERO

El módulo `core fs` proporciona operaciones para manejar ficheros.

Dentro de este módulo hay dos funciones para leer el contenido de un fichero (ambas leen el fichero completo):

- `readFileSync(fichero, opcs)`
Lectura síncrona.
- `readFile(fichero, opcs, callback)`
Lectura asíncrona.

LECTURA SÍNCRONA

```
"use strict";
const fs = require("fs");
try {
  const contenido = fs.readFileSync("fichero.txt",
    { encoding: "utf-8" });
  console.log("Fichero leído correctamente:");
  console.log(contenido);
} catch (err) {
  console.log("Se ha producido un error:");
  console.log(err.message);
}
```

```
ODEJS> node sincronoVSasincrono.js
```

```
Fichero leído correctamente:
```

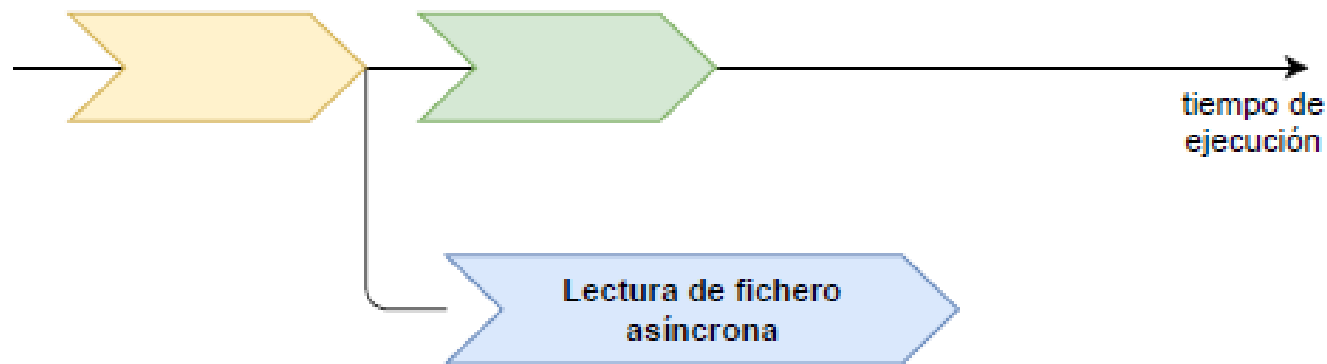
```
Esto es un fichero de texto para comprobar su lectura de forma síncrona y asíncrona
```

LECTURA SÍNCRONA

Las operaciones síncronas:

- **Bloquean** la ejecución del programa mientras se realizan.
- **Devuelven** el resultado de la operación.
- En caso de error, lanzan **excepciones**.

LECTURA ASÍNCRONA



Si la operación de lectura asíncrona se realiza de manera ajena al hilo de ejecución principal del programa, ¿cómo sabemos cuándo ha finalizado?

Mediante el uso de funciones **callback**.

LECTURA ASÍNCRONA

FUNCIONES CALLBACK

Una función **callback** es una función definida por el programador, pero que no está pensada para ser llamada directamente por este, sino por otro componente del sistema.

FUNCIÓN CALLBACK DE `readFile`

La función asíncrona `readFile` (módulo `fs`), además de recibir el nombre del fichero y las opciones de lectura, recibe un tercer parámetro, que es una función.

Esta función es definida por el programador, pero será llamada por `readFile` cuando la lectura del fichero haya finalizado.

Es, por tanto, una función callback.

FUNCIÓN CALLBACK DE `readFile`

A su vez, la función callback recibe dos parámetros:

- Un objeto de la clase `Error`, en el caso en que la lectura haya fallado.

Si no ha fallado, toma el valor `null`.

- El contenido del fichero leído, en el caso en que se haya leído correctamente.

Si ha fallado, toma el valor `undefined`.

FUNCIÓN CALLBACK DE readFile

```
fs.readFile("fichero.txt", { encoding:"utf-8" }, ficheroLeido );  
// Función callback  
function ficheroLeido (err, contenido) {  
    if (err) {  
        console.log("Se ha producido un error:");  
        console.log(err.message);  
    } else {  
        console.log("Fichero leído correctamente:");  
        console.log(contenido);  
    }  
}
```

Tras la lectura, **readFile** llamará a la función **ficheroLeido**.

FUNCIÓN CALLBACK DE readFile

Las funciones callback pueden definirse directamente como funciones anónimas (es lo más habitual). Está permitido utilizar la notación flecha.

```
"use strict";
const fs = require("fs");
fs.readFile("fichero.txt",
    { encoding: "utf-8" },
    function(err, contenido) {
        if (err) {
            console.log("Se ha producido un error:");
            console.log(err.message);
        } else {
            console.log("Fichero leído correctamente:");
            console.log(contenido);
        }
    }
);
```

LAS OPERACIONES ASÍNCRONAS

Las operaciones asíncronas:

- **No bloquean** la ejecución del programa.

Porque se ejecutan de manera concurrente.

- **No devuelven** ningún valor.

El resultado de la lectura se pasa como parámetro a la función *callback*.

- **No lanzan excepciones.**

En caso de error, es la función *callback* quien recibe la excepción como primer parámetro.

EL MODELO DE PROCESO DE NODE.JS

- Se ejecuta el hilo principal del programa insertándose las callbacks en una cola
- Cuando se termina el trabajo asociado a una callback, ésta se saca de la cola y se ejecuta

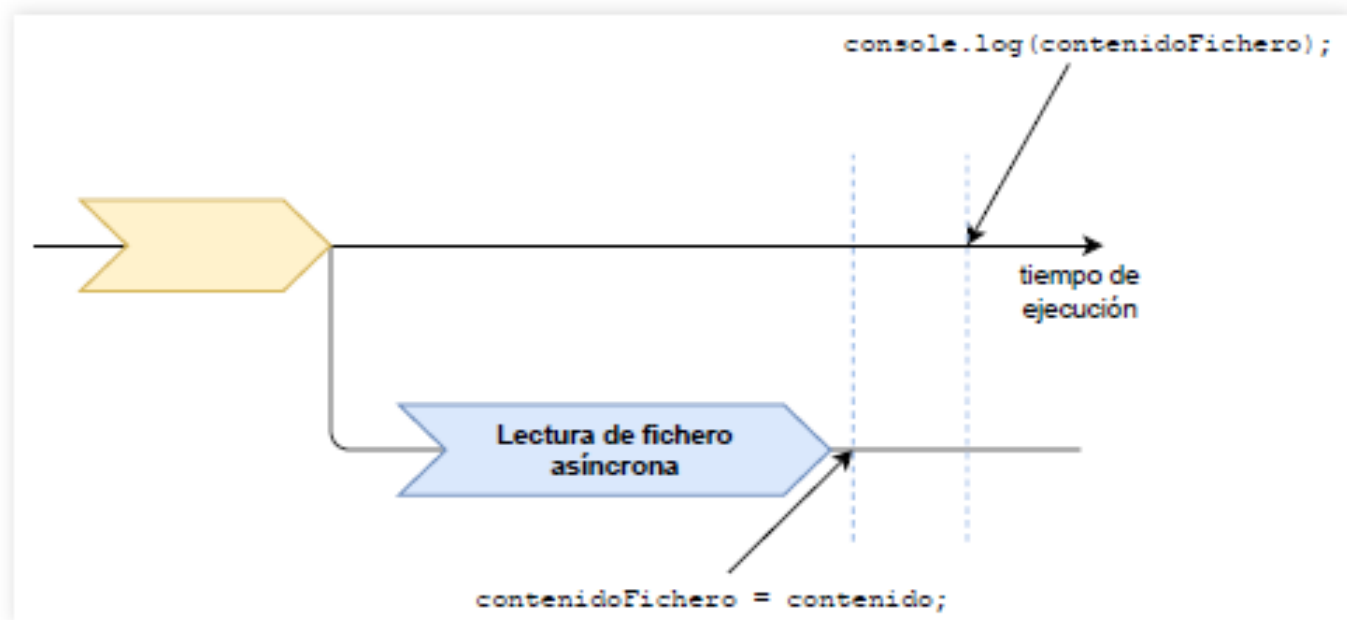
UN ERROR FRECUENTE

¡CUIDADO CON EL SIGUIENTE CÓDIGO...!

```
const fs = require("fs");
let contenidoFichero;
fs.readFile("prueba.js", {encoding: "utf-8"},
  function(err, contenido) {
    if (!err) {
      // Asignamos el contenido a la variable externa
      contenidoFichero = contenido;
    }
  });
console.log(contenidoFichero); // ¿Qué se imprime aquí?
```

UN ERROR FRECUENTE

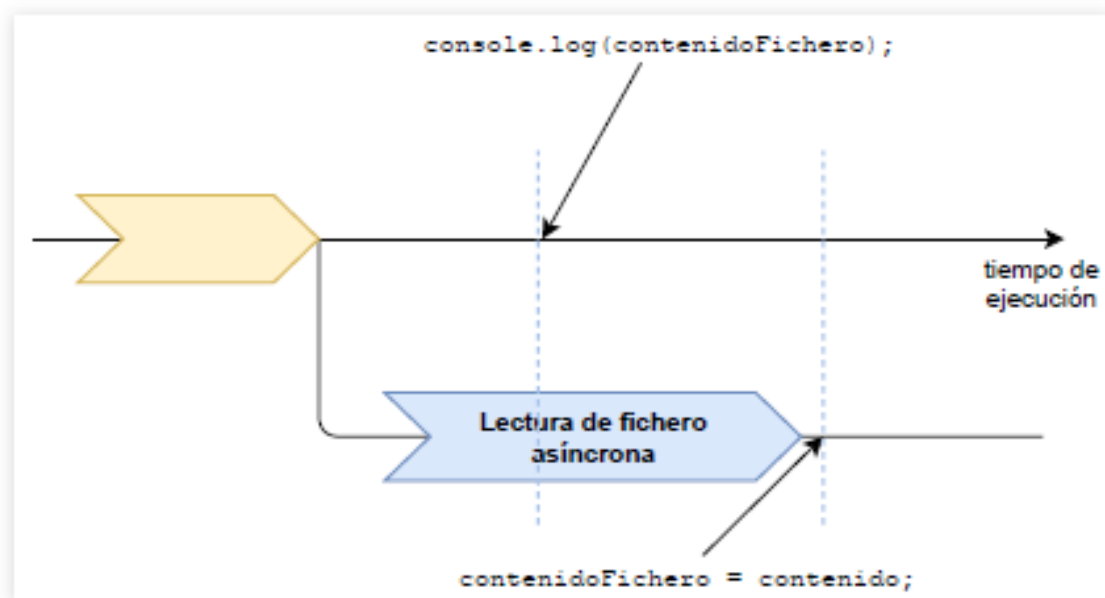
En el mejor de los casos, la llamada a `console.log` se ejecutaría después de que la lectura haya finalizado.



Pero esto no ocurre así nunca.

UN ERROR FRECUENTE

`console.log` se ejecuta antes de que la lectura termine:



En este caso se imprime `undefined`, ya que `contenidoFichero` no ha sido inicializada en el momento de ejecutar `console.log`.

UN ERROR FRECUENTE

MORALEJA

Si un fragmento del programa depende del resultado de una llamada asíncrona, no debe ir tras dicha llamada:

```
fs.readFile("fich.txt", function(err, contenido) {  
    ...  
});  
// hacer algo con el contenido
```

Debe ir dentro de la función callback:

```
fs.readFile("fich.txt", function(err, contenido) {  
    // hacer algo con el contenido  
    ...  
});
```

CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



MODULOS CORE: path y fs

- Variables y funciones predefinidas
- Nombres de fichero: módulo **path**
- Manejo de ficheros: módulo **fs**

VARIABLES PREDEFINIDAS

Las siguientes variables son accesibles desde cualquier módulo o script, pero no desde la *shell* de Node:

- **__filename**: Fichero .js que se está ejecutando (incl. directorio).
- **__dirname**: Directorio en el que se encuentra el fichero que se está ejecutando.

```
// globals.js  
// -----
```

```
console.log(__dirname);  
console.log(__filename);
```

MÓDULO `path`

Contiene funciones de utilidad para trabajar con nombres de ficheros de manera independiente del SO.

Funciones más importantes:

- `path.parse(filename)`
Obtiene las distintas componentes de `filename` (path, fichero, extensión, etc).
- `path.join(dir1, dir2, ...)` Concatena varios directorios utilizando el separador del sistema operativo: `/` en sistemas UNIX (Linux), `\` en Windows.

MÓDULO path

```
const path = require("path");
const infoFichero = path.parse(__filename)
console.log(infoFichero);

// {
  root: 'C:\\',
  dir: 'H:\\\\UCM\\\\FDI\\\\AW\\\\NODE\\\\ejercicios\\pruebas',
  base: 'ej1.js',
  ext: '.js',
  name: 'ej1'
}

const nuevoFichero = path.join(infoFichero.dir, "nuevo",
infoFichero.base);
console.log(nuevoFichero);
// H:\\\\UCM\\\\FDI\\\\AW\\\\NODE\\\\ejercicios\\pruebas\\nuevo\\ej1.js
```

MÓDULO fs

Contiene funciones relativas al sistema de ficheros.

- `readFile(fichero, options, callback)`

Lectura asíncrona de un fichero.

La función callback recibe como primer parámetro un objeto `Error` (si se produce error) y como segundo parámetro el contenido del fichero.

- `writeFile(fichero, datos, options, callback)`

Escritura asíncrona de un fichero.

La función callback recibe como único parámetro un objeto `Error` (si se produce error).

MÓDULO fs

Ejemplo del modelo de procesamiento de node.js

```
"use strict";

let fs=require('fs');

for (let i=1; i<10; i++) {
  let fichero = "f" + i + ".txt";
  console.log("Solicitada la escritura del fichero " + fichero);
  fs.writeFile(fichero,fichero,function(err) {
    if (!err) {
      console.log("Terminada la escritura del fichero" + fichero);
    }
  })
}
```

MÓDULO fs

Ejemplo de encadenamiento de funciones asíncronas para "sincronizarlas" (o serializarlas).

```
"use strict";
const fs = require("fs");

const texto = "Actualmente el registro de npm tiene 800.000 paquetes."

fs.writeFile("npm.txt", texto, {encoding: "utf-8"},
  function(err) {
    if (err) {
      console.log("Error al escribir el fichero.");
    } else {
      console.log("Fichero escrito correctamente.");
      fs.readFile("npm.txt", {encoding: "utf-8"},
        function(err, contenido) {
          if (!err) {
            console.log(contenido);
          }
        });
    }
  });
```

MÓDULO fs

```
let contenidoFichero;
fs.readFile("fichero.txt",
  { encoding: "utf-8" },
  function(err, contenido) {
    if (!err) {
      // Asignamos el contenido a la variable
      // externa
      contenidoFichero = contenido;
      console.log(contenidoFichero);
    } else {
      console.log("Error leyendo el fichero...");
    }
  });

console.log(path.parse(__filename));

// Escritura de un fichero:
for (let i=1; i<5; i++) {
  let fichero = "f" + i + ".txt";
  console.log("Solicitada la escritura del fichero " + fichero);
  fs.writeFile(fichero, fichero, function(err) {
    if (!err) {
      console.log("Terminada la escritura del fichero" + fichero);
    }
  });
}
```

MÓDULO fs

```
{  
  root: 'C:\\',  
  dir: 'C:\\Users\\....',  
  base: 'ficheros.js',  
  ext: '.js',  
  name: 'ficheros'  
}  
Solicitada la escritura del fichero f1.txt  
Solicitada la escritura del fichero f2.txt  
Solicitada la escritura del fichero f3.txt  
Solicitada la escritura del fichero f4.txt  
Terminada la escritura del fichero f2.txt  
Terminada la escritura del fichero f4.txt  
Esto es un fichero de texto para comprobar su lectura  
Terminada la escritura del fichero f1.txt  
Terminada la escritura del fichero f3.txt
```

MÓDULO fs

Si no se indica la opción **encoding**, estas funciones trabajan con objetos de tipo **Buffer**, en lugar de cadenas. Un buffer representa una secuencia de bytes.

Esto sirve para trabajar con ficheros binarios.

```
console.log("Lectura binaria de un fichero");
fs.readFile("ejemplo.pdf", function(err, buffer) {
  if (!err) {
    // Obtenemos los cuatro primeros bytes
    const mark = buffer.slice(0,4);
    // Y los imprimimos con la codificación ASCII
    console.log("mark: " + mark.toString("ascii"));
    // console.log("buffer: " + buffer);
  } else {
    console.log("error en la lectura binaria");
  }
});
```

MÓDULO fs

OTRAS FUNCIONES DE fs

- Leer y escribir de manera segmentada: `open`, `close`, `read`, `write`.
- Manejo del sistema de ficheros: `mkdir`, `fstat`, `rename`, `unlink`, etc.

Todas estas funciones son **asíncronas**.

Más información:

<https://nodejs.org/dist/latest-v14.x/docs/api/fs.html>

MÓDULO fs: VARIANTES SÍNCRONAS

Son bloqueantes y (posiblemente) devuelven un resultado, en lugar de utilizar funciones callback.

- `readFileSync`, `writeFileSync`
- `openSync`, `closeSync`, `readSync`, `writeSync`.
- `mkdirSync`, `fstatSync`, `renameSync`, etc.

No se utilizarán en esta asignatura.

CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



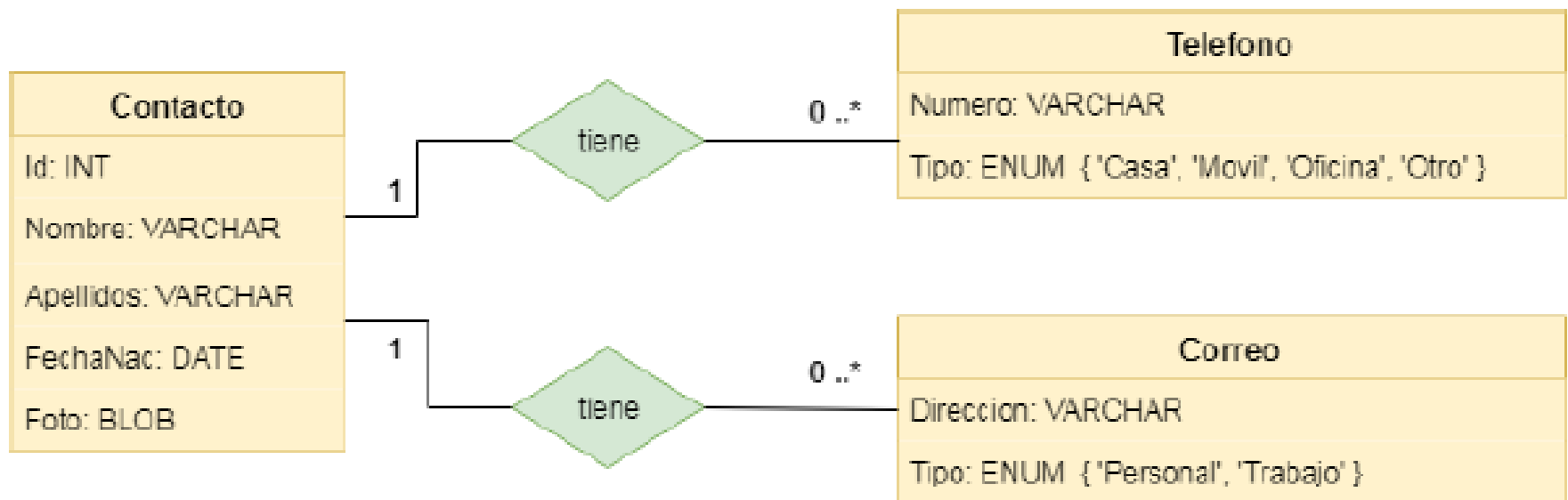
ACCESO A BASES DE DATOS

Paquetes para conexión a un SGBD:

- Bases de datos **relacionales**:
 - **mysql**: MySQL, MariaDB. **Este tema**
 - **node-oracledb**: Oracle.
 - **sequelize**: ORM para MySQL.
- Bases de datos **documentales**:
 - **mongodb**: MongoDB (JSON)
 - **mongoose**: ODM para MongoDB
 - **node-exist**: eXist-db (XML)
 - **basex**: BaseX (XML)

EJEMPLO DE BASE DE DATOS

Agenda de contactos



EJEMPLO DE BASE DE DATOS

Tabla Contactos

Id	Nombre	Apellidos	FechaNac	Foto
1	Javier	Fernández Montoro	1980-03-21	NULL
2	Estela	Hernández de la Fuente	1979-03-15	[BLOB - 11.7 KiB]
3	Gloria	Ruiz Sánchez	1983-10-01	[BLOB - 10.4 KiB]
4	Fermín	Córdoba Moreno	NULL	NULL

Tabla Telefonos

Id	Numero	Tipo
1	606320123	Móvil
2	680200100	Móvil
1	916140583	Casa
4	916151032	Oficina

Tabla Correos

Id	Correo	Tipo
1	androx80@hotmail.com	Personal
3	gloria.martinez@atm.es	Trabajo
1	javier.fernandez@gmail.com	Trabajo
1	jfm@telefonica.es	Trabajo
2	wall_es@yahoo.es	Personal

PAQUETE mysql

Instalación mediante **npm**

```
npm install mysql --save
```

PASOS PARA ACCEDER A UNA BASE DE DATOS

1. Crear e inicializar un objeto **Connection**.
2. Realizar la conexión.
3. Hacer la operación (consulta o modificación).
4. Cerrar la conexión.

Salvo la primera, todas las operaciones son **asíncronas**.

INICIALIZAR UNA CONEXIÓN

En el paquete `mysql` existe la función `createConnection()` para crear e inicializar la conexión (paso 1).

Para los pasos 2, 3 y 4 existen los métodos `connect()`, `query()` y `end()`.

Pero nosotros no vamos a abrir y cerrar conexiones de manera individual.

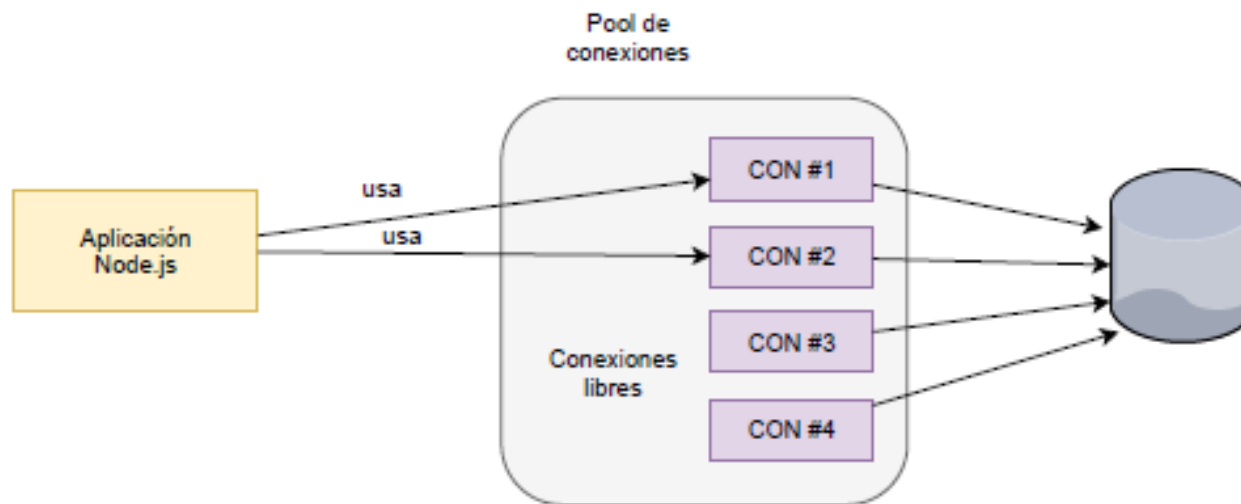
En teoría, una aplicación web debería abrir y cerrar una conexión cada vez que atienda una petición que requiera acceso a la BD, pero la creación de conexiones es un proceso relativamente **costoso**.

POOL DE CONEXIONES

Un **pool de conexiones** es un contenedor de conexiones permanentemente abiertas a una determinada BD

Con un pool conseguimos que las conexiones se reutilicen en sucesivas consultas a la BD, sin necesidad de crear conexiones nuevas continuamente.

POOL DE CONEXIONES



Cuando se quiere realizar una consulta a la BD, se adquiere una conexión libre del *pool* de conexiones.

Cuando se ha terminado de trabajar con la conexión, se devuelve al *pool*, en lugar de cerrarla.

POOL DE CONEXIONES

El paquete `mysql` permite la creación de pools con el método **síncrono** `createPool()` que devuelve un objeto de la clase `Pool`.

```
const mysql = require("mysql");
const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "",
  database: "mibd"
});

//...
```

Los objetos `Pool` permiten obtener conexiones mediante el método **asíncrono** `getConnection()`.

OBTENER UNA CONEXIÓN DEL POOL

Método `getConnection(callback)`

```
pool.getConnection(function(err, connection) {  
  if (err) {  
    console.log('Error al obtener la conexión: ${err.message}');  
  } else {  
    // .... realizar consulta ....  
  }  
});
```

- El segundo parámetro de la callback es un objeto `Connection`.
- Con el objeto `Connection` se puede ejecutar consultas y modificaciones mediante SQL.

EJECUTAR CONSULTAS

Método `query(consulta, callback)`

La función `callback` recibe, además del objeto `error`, el resultado de la consulta.

```
connection.query(  
  "SELECT nombre, apellidos FROM contactos",  
  function (err, filas) {  
    if (err) {  
      console.log('Error en la consulta a la base de datos');  
    }  
    else {  
      // Acceso a las filas resultado de la consulta  
    }  
  }  
);
```

EJECUTAR CONSULTAS

El resultado de una consulta es un array de objetos.

Cada objeto representa una fila. Los atributos de cada objeto son los nombres de las columnas devueltos por **SELECT**. Los valores asociados a cada atributo dependen del tipo de la columna correspondiente:

Tipo Node	Tipo SQL
number	INT, SMALLINT, FLOAT, DOUBLE, ...
string	CHAR, VARCHAR, TEXT, ENUM, ...
Date	TIMESTAMP, DATE, DATETIME
Buffer	BLOB, BINARY, DATETIME, ...

EJECUTAR CONSULTAS

```
connection.query(  
  "SELECT Nombre, Apellidos FROM Contactos",  
  function(err, filas) {  
    if (err) {  
      console.log('Error en la consulta a la base de datos');  
    }  
    else {  
      // Acceso a las filas resultado de la consulta  
      filas.forEach(function(fila) {  
        console.log(`${fila.Nombre} ${fila.Apellidos}`);  
      });  
    }  
  }  
);
```

Felipe Lotas
Alberto Cadiscos
Borja Mon de York
Ernesto Esvida

EJECUTAR CONSULTAS

El pool contiene un número limitado de conexiones.

Por tanto, conviene devolver la conexión al pool, mediante el método `release()`, una vez obtenidos los resultados de la consulta.

```
connection.query(  
  "SELECT nombre, apellidos FROM contactos",  
  function (err, filas) {  
    connection.release();  
    if (err) {  
      console.log('Error en la consulta a la base de datos');  
    }  
    else {  
      // Acceso a las filas resultado de la consulta  
      filas.forEach(function(fila) {  
        console.log(`${fila.nombre} ${fila.apellidos}`)  
      });  
    }  
  }  
);
```

EJECUTAR CONSULTAS

```
const mysql = require("mysql");
const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "",
  database: "mibd"
});
pool.getConnection(function(err, connection) {
  if (err) {
    console.log('Error al obtener la conexión: ${err.message}');
  } else {
    connection.query(
      "SELECT nombre, apellidos FROM contactos",
      function (err, filas) {
        connection.release();
        if (err)
          console.log('Error en la consulta a la base de datos');
        else {
          filas.forEach(function(fila) {
            console.log(`${fila.nombre} ${fila.apellidos}`)
          })
        }
      }
    );
  }
});
```


INSERCIÓN, BORRADOS Y ACTUALIZACIONES

Al igual que en las consultas, se utiliza `query()`.

En estos casos, la función callback recibe un objeto con información sobre el resultado de la operación.

Atributos del objeto resultado:

- **affectedRows**: número de filas insertadas, actualizadas o borradas.
- **insertId**: identificador de la última fila insertada, en caso de ser de tipo **AUTO_INCREMENT**.

EJEMPLO

```
pool.getConnection(function(err, connection) {
  if (err) {
    console.log('Error al obtener la conexión: ${err.message}');
  } else {
    const sql = "INSERT INTO contactos(nombre, apellidos) "
      + "VALUES ('Diana', 'Díaz')",
    connection.query(sql, function (err, filas) {
      connection.release();
      if (err) {
        console.log('Error en la consulta a la base de datos');
      }
      else {
        // Imprime el identificador de la nueva fila
        console.log(resultado.insertId)
        // Imprime el número de filas insertadas
        console.log(resultado.affectedRows);
      }
    })
  }
});
```

CONEXIONES ÚNICAS vs POOL DE CONEXIONES

Conexión única

```
conexion = mysql.CreateConnection( ... )
```

```
conexion.connect( ... )
```

```
conexion.query( ... )
```

```
conexion.end( ... )
```

Conexión en pool

```
pool = mysql.CreatePool( ... )
```

```
pool.getConnection( ... ) → conexion
```

```
conexion.query( ... )
```

```
conexion.release( ... )
```

```
pool.end( ... )
```

CONSULTAS SQL PARAMETRIZADAS

-

Supongamos que queremos realizar un programa que solicite un identificador al usuario y muestre la entrada de la tabla **contactos** con ese identificador.

```
Introduce un identificador: 1  
[ RowDataPacket { Nombre: 'Javier', Apellidos: 'Fernández Montoro' } ]
```

CONSULTAS SQL PARAMETRIZADAS

```
// Suponemos que la variable 'id' contiene el identificador
// introducido por el usuario
pool.getConnection(function(err, connection) {
  if (err) {
    console.log('Error al obtener la conexión: ${err.message}');
  }
  else {
    const sql='SELECT Nombre, Apellidos FROM Contactos WHERE Id=${data.id}';
    connection.query(sql,
      function(err, filas) {
        connection.release();
        if (err)
          console.log("Error en la consulta");
        else
          console.log(filas);
      });
  }
});
```

La variable `id` proviene de la entrada introducida por el usuario. Esta variable pasa directamente a la consulta SQL.

¿Qué pasa si el usuario hubiese introducido `1 OR TRUE`?

CONSULTAS SQL PARAMETRIZADAS

Introduce un identificador: 1 OR TRUE

```
[ RowDataPacket { Nombre: 'Javier', Apellidos: 'Fernández Montoro' },  
  RowDataPacket { Nombre: 'Estela', Apellidos: 'Hernández de la Fuente' },  
  RowDataPacket { Nombre: 'Gloria', Apellidos: 'Ruiz Sánchez' },  
  RowDataPacket { Nombre: 'Fermín', Apellidos: 'Córdoba Moreno' } ]
```

Sentencia SQL ejecutada:

```
SELECT Nombre, Apellidos FROM contactos WHERE Id = 1 OR TRUE
```

CONSULTAS SQL PARAMETRIZADAS

INYECCIÓN SQL



Fuente: <https://xkcd.com/327/>

CONSULTAS SQL PARAMETRIZADAS

Una **consulta paramétrica** es una consulta SQL con «huecos» (denominados marcadores), representados mediante el símbolo **?**.

Al rellenar los marcadores con valores concretos, estos se «escapean» de manera que se previenen los ataques de inyección SQL.

Ejemplo de consulta paramétrica:

```
INSERT INTO contactos (Nombre, Apellidos) VALUES (?, ?);
```


CONSULTAS SQL PARAMETRIZADAS

El método `query()` puede recibir una consulta paramétrica. En este caso, el segundo parámetro de este método recibe un array con los valores asignados a los marcadores.

Por ejemplo, si se ejecuta la consulta paramétrica,

```
SELECT Nombre, Apellidos FROM contactos  
WHERE Id = ?
```

y asignamos la cadena `"1 OR TRUE"` al marcador, en realidad se ejecutará la siguiente consulta:

```
SELECT Nombre, Apellidos FROM contactos  
WHERE Id = '1 OR TRUE'
```

CONSULTAS SQL PARAMETRIZADAS

El método `query()` con consultas paramétricas:

`query(consulta, valores_marcadores, callback)`

```
// Suponemos que la variable `id` contiene el identificador
// introducido por el usuario
connection.query(
  `SELECT Nombre, Apellidos FROM contactos WHERE Id = ?`,
  [id],
  function(err, rows) {
    if(err){
      console.log('Error en la consulta');
    }
    else {
      console.log(rows);
    }
  });
```

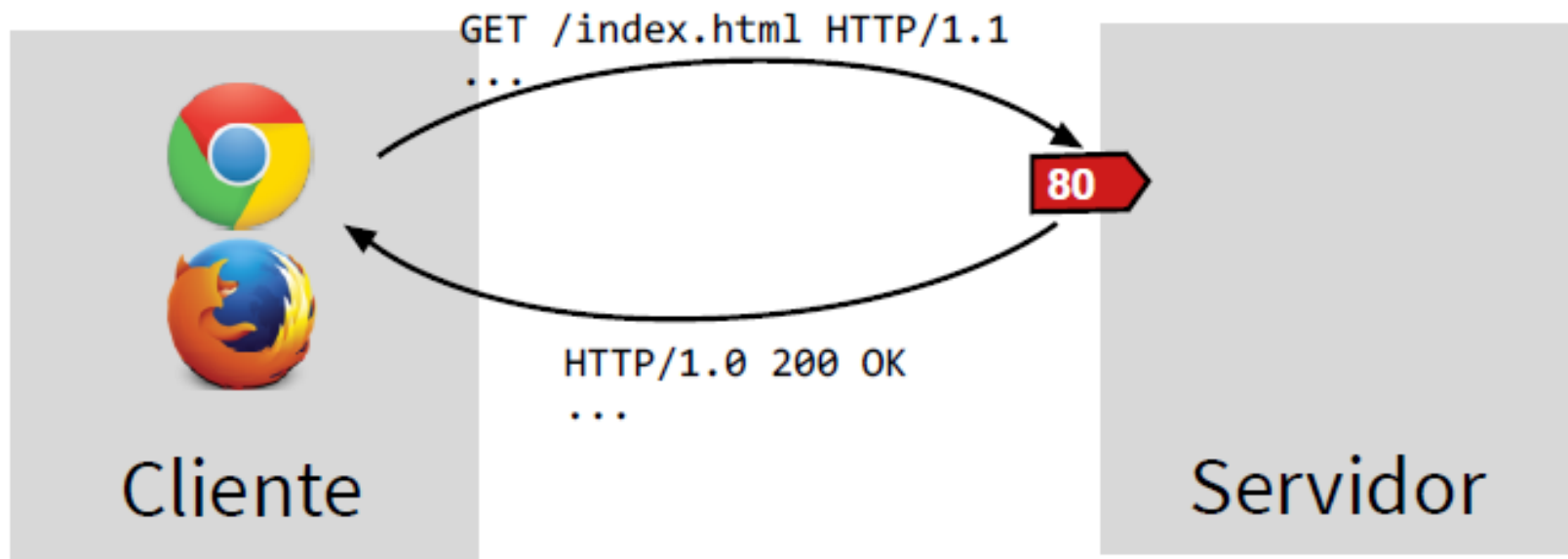
CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- BIBLIOGRAFÍA



SERVIDORES WEB CON NODE

El modelo cliente-servidor aplicado a la web.



El **cliente** es un navegador web.

El **servidor** es un programa que está escuchando en el puerto TCP 80.

SERVIDORES WEB CON NODE

Cuando el usuario introduce una URL en el **cliente** se envía una **petición HTTP al servidor** correspondiente.

```
GET /register.html HTTP/1.1
```

Línea de petición

```
Host: www.foo.com
```

```
Accept: text/html
```

```
Accept-Language: en-US, es-ES
```

Cabeceras

```
Connection: keep-alive
```

```
If-Modified-Since: Fri, 26 Feb 2016 03:09:05 GMT
```

(línea en blanco)

```
name=Francisco%20Sanchez&age=12
```

Cuerpo de la petición (opcional)

```
...
```

LÍNEA DE PETICIÓN

Tiene el siguiente formato:

```
[método] [recurso] [versión_HTTP]
```

- **[método]**: Acción a realizar (GET, POST, HEAD, PUT, ...)
- **[recurso]**: Nombre del recurso (URI) sobre el que se accederá.
- **[versión_HTTP]**: HTTP/1.0 o HTTP/1.1.

Ejemplo:

```
GET /register.html HTTP/1.1
```

SERVIDORES WEB CON NODE

El **servidor** analiza la petición y envía una **respuesta** al **cliente**.

```
HTTP/1.1 200 OK
```

Línea de estado

```
Date: Mon, 15 Aug 2016 14:05:30 GMT
```

```
Content-Type: text/html
```

Cabeceras de la respuesta

```
...
```

(línea en blanco)

```
<!DOCTYPE html>
```

```
<html lang="es">
```

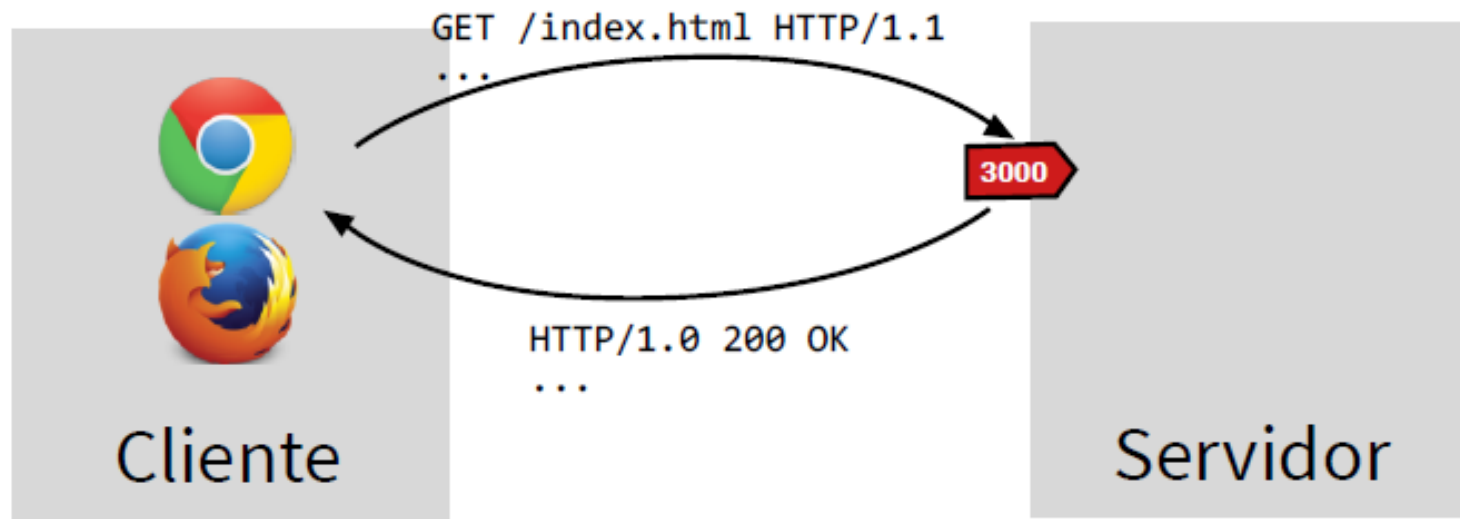
```
<head>
```

```
  <title>...</title>
```

```
...
```

Cuerpo de la respuesta

SERVIDORES WEB CON NODE



Implementaremos la funcionalidad del **servidor**.

En la fase de desarrollo de una aplicación web se suele utilizar otro puerto distinto del 80. Utilizaremos el puerto 3000.

SERVIDORES WEB CON NODE

La creación de servidores requiere el uso de *sockets*.

Para ello, deberíamos:

- Crear un socket asociado al puerto 80 (o cualquier otro).
- Aceptar peticiones de clientes.
- Analizar sintácticamente la petición HTTP del cliente, que viene como cadena de texto.
- Generar una respuesta en formato HTTP.

Node dispone del módulo `http` para implementar servidores web.

EL MÓDULO `http`

```
const http = require("http");
```

Los objetos de la clase `Server` representan servidores web, y son creados mediante la función `createServer()`, que recibe una función callback.

```
const servidor = http.createServer(function (request, response) {  
    ...  
});
```

La función callback es definida por el programador, y llamada cada vez que algún cliente realiza una petición HTTP al servidor.

EL MÓDULO `http`

```
http.createServer(function(request, response) { /* ... */ });
```

El parámetro `request` de esta función callback contiene un objeto de la clase `IncomingMessage`, con atributos y métodos para acceder a cada componente de la petición: método, URL, cabeceras, etc.

El parámetro `response` (clase `ServerResponse`) tiene métodos y atributos que especifican la respuesta HTTP que se devolverá al cliente.

EL MÓDULO `http`

La función `createServer()` tan solo sirve para establecer la función *callback* que procesa las peticiones, pero no inicia el servidor en ningún puerto.

Para **iniciar el servidor** se utiliza el método `listen()` del servidor recién creado, indicando el puerto en el que va a escuchar y (opcionalmente) una función *callback* que será llamada cuando comience la escucha.

EL MÓDULO http

```
"use strict";
const http = require("http");

// Establecimiento de la función callback del servidor
const servidor = http.createServer(function(request, response) {
    // ...
});

// Inicio del servidor
servidor.listen(3000, function(err) {
    if (err) {
        console.log(`Error al abrir el puerto 3000: ${err}`);
    } else {
        console.log("Servidor escuchando en el puerto 3000.");
    }
});
```

EL OBJETO request

Atributos más importantes de **request**:

- **method**: (string) método HTTP utilizado (GET, POST, PUT, etc.).
- **url**: (string) URL a la que se accede, excluyendo el nombre del servidor.
- **headers**: Objeto con las cabeceras de la petición.

Se pueden mostrar por consola los atributos:

```
const servidor = http.createServer(function(request, response) {  
  console.log(`Método: ${request.method}`);  
  console.log(`URL: ${request.url}`);  
  console.log(request.headers);  
});
```

EJEMPLO

SERVIDOR QUE MUESTRA SUS PETICIONES

1. Iniciar un servidor en el puerto 3000.

```
const http = require('http');

//Establecimiento de la función callback del servidor
let servidor = http.createServer(function(request, response) {
  // funcionalidad del servidor
  console.log('Método: ${request.method}');
  console.log('URL: ${request.url}');
  console.log(request.headers);
});

// Inicio del servidor
servidor.listen(3000, function(err) {
  if (err)
    console.log("Error al iniciar el servidor");
  else
    console.log("Servidor escuchando en el puerto 3000");
});
```

EJEMPLO

2. Iniciar el navegador e introducir la siguiente dirección:

```
http://localhost:3000/index.html
```

El servidor imprime un mensaje similar al siguiente:

```
Método: GET
URL:/index.html
{
  host: 'localhost:3000',
  connection: 'keep-alive',
  'cache-control': 'max-age=0',
  'sec-ch-ua': '"Chromium";v="118", "Google Chrome";v="118", "Not=A?Brand";v="99"',
  'sec-ch-ua-mobile': '?0',
  'sec-ch-ua-platform': '"Windows"',
  'upgrade-insecure-requests': '1',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36',
  ...
}
```


EL OBJETO response

El objeto `response` de la función callback permite responder al servidor. Sus atributos y métodos más importantes son:

- `statusCode`: indica el código de respuesta: 200, 404, etc.
- `setHeader(clave, valor)`: añade una cabecera a la respuesta.
- `write(str)`: envía la cadena `str` en el cuerpo de la respuesta.
- `end([str])`: delimita el final de la respuesta (obligatorio).

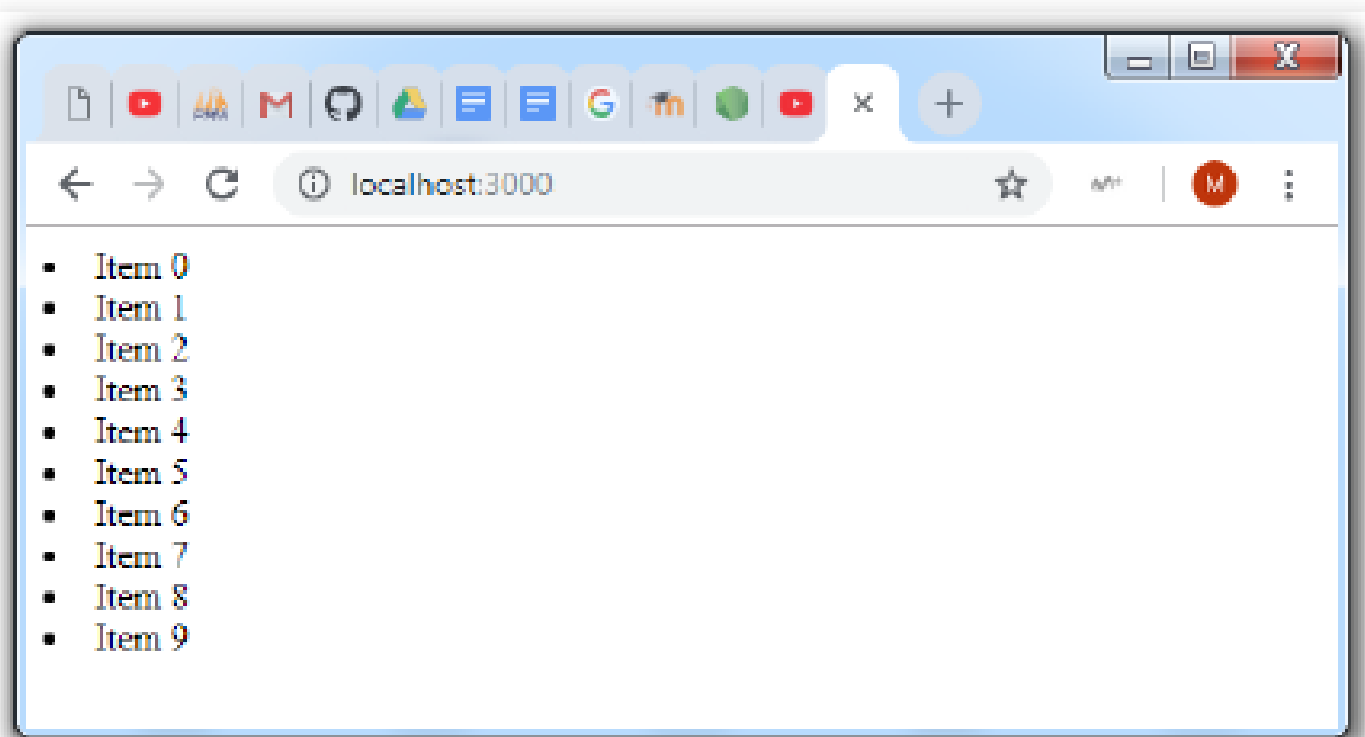
EJEMPLO

SERVIDOR QUE DEVUELVE UNA PÁGINA CON UNA LISTA

```
const servidor = http.createServer(function(request, response) {  
  response.statusCode = 200;  
  response.setHeader("Content-Type", "text/html");  
  response.write('<!DOCTYPE html>');  
  response.write('<html>');  
  response.write('<head>');  
  response.write('<title>Página de prueba</title>');  
  response.write('<meta charset="utf-8">');  
  response.write('</head>');  
  response.write('<body></ul>');  
  for (let i = 0; i < 10; i++) {  
    response.write('<li>Item ${i}</li>');  
  }  
  response.write('</ul></body></html>');  
  response.end();  
});
```

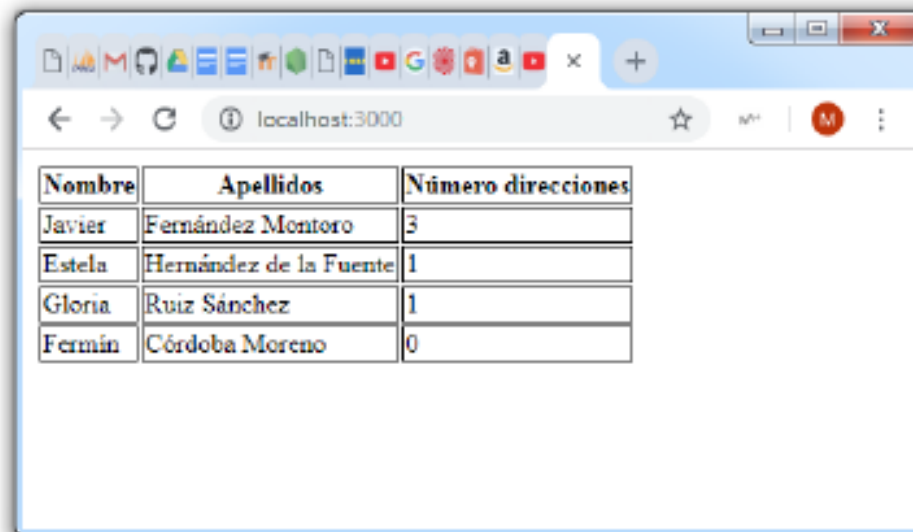
EJEMPLO

Al acceder a `http://localhost:3000`.



EJEMPLO: ACCESO A UNA BASE DE DATOS

Utilizando la base de datos "Contactos" vista anteriormente, el servidor devuelve una página con una tabla que contiene para cada usuario, su nombre, apellidos y el número de direcciones de correo asociadas.

A screenshot of a web browser window. The address bar shows 'localhost:3000'. The page content is a table with three columns: 'Nombre', 'Apellidos', and 'Número direcciones'. The table contains five rows of data.

Nombre	Apellidos	Número direcciones
Javier	Fernández Montoro	3
Estela	Hernández de la Fuente	1
Gloria	Ruiz Sánchez	1
Fermin	Córdoba Moreno	0

EJEMPLO: ACCESO A UNA BASE DE DATOS

1. Se implementa la función `consultaBD()`, que realiza la consulta SQL correspondiente y pasa los resultados a la función callback pasada como parámetro.

```
const http = require('http');
const mysql = require("mysql");
const pool = mysql.createPool({...});

function consultaBD(callback) {
  pool.getConnection(function(err, conexion) {
    if (err) {callback(err);}
    else {
      conexion.query(
        "SELECT Nombre, Apellidos, COUNT(corr.Correo) as NumCorreos "
        + "FROM Contactos con "
        + "LEFT JOIN Correos corr ON con.Id = corr.Id"
        + "GROUP BY con.id", function(err, rows) {
          conexion.release();
          if (err) { callback(err); }
          else { callback(null, rows); }
        });
    }
  });
}
```

EJEMPLO: ACCESO A UNA BASE DE DATOS

2. Se programa la función callback del servidor que invoca a la función `consultaBD()`.

En el callback de la función `consultaBD()`, si no hay error, se invoca a la función `devolverPagina()` que construye y envía la página al cliente.

```
const servidor = http.createServer(function(request, response) {  
  consultaDB(function(err, filasBD) {  
    if (err) {  
      response.statusCode = 500; // Internal Server Error  
      console.error(err);  
    } else {  
      response.statusCode = 200; // No ha habido errores.  
      devolverPagina(response, filasBD);  
    }  
  });  
});
```

EJEMPLO: ACCESO A UNA BASE DE DATOS

3. La función `devolverPagina()` genera el código HTML que se devuelve al cliente.

```
function devolverPagina(response, filasBD) {
    response.setHeader("Content-Type", "text/html");
    response.write('<html>');
    response.write('<head>');
    response.write('<title>Base de datos de teléfonos</title>');
    response.write('<meta charset="utf-8">');
    response.write('<style>th, td {border: 1px solid} </style>');
    response.write('</head>');
    response.write('<body>');
    response.write('<table>');
    response.write('<tr><th>Nombre</th><th>Apellidos</th>'
        + '<th>Número de direcciones</th></tr>');
    filasBD.forEach(function(fila) {
        response.write('<tr>');
        response.write('<td>${fila.Nombre}</td>');
        response.write('<td>${fila.Apellidos}</td>');
        response.write('<td>${fila.NumCorreo}</td>');
        response.write('</tr>');
    });
    response.write('</table>');
    response.write('</body></html>');
    response.end();
}
```

ESQUEMA GENERAL DE UN SERVIDOR

La función callback del servidor devuelve una respuesta en función de la petición recibida.

Una función callback típica tendría la siguiente forma:

```
const servidor = http.createServer(function(request, response) {  
  const method = request.method;  
  const url = url.parse(request.url, true); // "objeto" url  
  const pathname = url.pathname; // último elemento de la cadena de la url  
  const query = url.query; // objeto query  
  if(method === 'GET' && pathname === '/index.html') {  
    // Servir la página principal  
  } else if(method === 'GET' && pathname === '/index.css') {  
    // Servir hoja de estilo  
  } else if (...) {...}  
  //...  
  //...  
} else {  
  response.statusCode = 404;  
}  
});
```


ESQUEMA COMPLICADO

La función **callback** del servidor tiene que gestionar:

- Generación de páginas web complejas mediante **response.write()**.
- Procesamiento de **formularios: url y querystring**.
- Envío de ficheros al servidor.
- Manejo de cookies, sesiones, etc.

El módulo **http** es de demasiado bajo nivel para estas tareas. Es recomendable el uso de un **framework** que abstraiga los detalles técnicos.

Express.js

CONTENIDO

- INTRODUCCIÓN
- MÓDULOS Y PAQUETES
- EL MODELO ASÍNCRONO
- MÓDULOS CORE
- ACCESO A BASES DE DATOS
- SERVIDORES WEB
- **BIBLIOGRAFÍA**



BIBLIOGRAFIA

- B.A. Syed
Beginning Node.js
Apress, 2014
- S. Powers
Learning Node, 2nd edition
O'Reilly (2016)
- Documentación de las librerías de Node

<https://nodejs.org/docs/latest-v12.x/api/>

